



UNIVERSIDADE
FEDERAL DO CEARÁ

Relatório Final

Manutenção De Software - 01 - 2026.1

Profa. Carla Ilane Moreira Bezerra

Autores

FRANCISCO SAMUEL CABRAL LEITÃO

WESLEY LESSA PINHEIRO

Sumário

Sumário	1
1. Caracterização do Projeto	3
2. Métricas Antes da Refatoração	3
2.1 Radon Antes da Refatoração	3
2.1.1 Complexidade Ciclomática por Arquivo Antes da Refatoração	3
2.1.2 Complexidade Ciclomática por Função Antes da Refatoração	4
2.1.3 Índice de Manutenibilidade Antes da Refatoração	5
2.1.4 Métricas brutas Antes da Refatoração	5
2.2 CodeCarbon Antes da Refatoração	5
2.3 Pylint Antes da Refatoração	6
2.3.1 Distribuição do Smells Antes da Refatoração	6
2.3.2 Distribuição dos Problemas de Código Antes da Refatoração	7
2.3.3 Arquivos mais Críticos Antes da Refatoração	8
2.3.4 Score Projeto Antes da Refatoração	8
2.4 Pytest Antes da Refatoração	8
2.4.1 Cobertura de testes Antes da Refatoração	8
2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração	13
3. Processo de Refatoração	13
3.1 too-many-statements	13
3.1.1 Ocorrência Número 1	13
3.1.2 Ocorrência Número 2	14
3.1.3 Ocorrência Número 3	16
3.1.4 Ocorrência Número 4	17
3.1.5 Ocorrência Número 5	19
3.1.6 Ocorrência Número 6	20
3.1.7 Ocorrência Número 7	22
3.2 too-many-instance-attributes	23
3.2.1 Ocorrência Número 1	23
3.2.2 Ocorrência Número 2	25
3.2.3 Ocorrência Número 3	26
3.2.4 Ocorrência Número 4	28
3.2.5 Ocorrência Número 5	29
3.3 too-many-branches	31
3.3.1 Ocorrência Número 1	31
3.3.2 Ocorrência Número 2	33
3.3.3 Ocorrência Número 3	35
3.3.4 Ocorrência Número 4	37
3.3.5 Ocorrência Número 5	39
3.3.6 Ocorrência Número 6	41

3.3.7 Ocorrência Número 7	43
3.3.8 Ocorrência Número 8	45
3.3.9 Ocorrência Número 9	47
3.3.10 Ocorrência Número 10	49
3.3.11 Ocorrência Número 11	51
3.3.12 Ocorrência Número 12	53
3.3.13 Ocorrência Número 13	55
3.3.14 Ocorrência Número 14	57
3.3.15 Ocorrência Número 15	59
3.3.16 Ocorrência Número 16	61
3.3.17 Ocorrência Número 17	63
3.3.18 Ocorrência Número 18	65
3.3.19 Ocorrência Número 19	67
3.3.20 Ocorrência Número 20	69
3.3.21 Ocorrência Número 21	71
3.3.22 Ocorrência Número 22	73
4. Métricas Após a Refatoração	75
4.1 Radon Após a Refatoração	75
4.1.1 Complexidade Ciclomática por Arquivo Após a Refatoração	75
4.1.2 Complexidade Ciclomática por Função Após a Refatoração	75
4.1.3 Índice de Manutenibilidade Após a Refatoração	76
4.1.4 Métricas Brutas Após a Refatoração	76
4.2 CodeCarbon Após a Refatoração	76
4.3 Pylint Após a Refatoração	77
4.3.1 Distribuição do Smells Após a Refatoração	77
4.3.2 Distribuição dos Problemas de Código Após a Refatoração	77
4.3.3 Arquivos mais Críticos Após a Refatoração	77
4.3.4 Score Projeto Após a Refatoração	77
4.4 Pytest Após a Refatoração	77
4.4.1 Cobertura de testes Após a Refatoração	77
4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração	77
5. Comparação dos Resultados	78
6. Percepções sobre a Prática	78
7. Links	78

1. Caracterização do Projeto

Nome Projeto: Great Tables

O Great Tables é uma biblioteca que possibilita a qualquer pessoa criar tabelas visualmente incríveis em Python. A filosofia do projeto é permitir a construção de uma ampla variedade de tabelas úteis a partir de um conjunto coeso de componentes estruturais. É possível combinar elementos como cabeçalho e rodapé, anexar um stub (para rótulos de linhas), organizar rótulos

superiores (spanners) sobre as colunas e muito mais. Além disso, a biblioteca oferece recursos avançados para a formatação dos valores das células.

As principais capacidades incluem:

- **Estruturação modular:** flexibilidade para combinar cabeçalhos, rodapés, colunas e rótulos de linha de forma personalizada;
- **Organização avançada de dados:** suporte para agrupar colunas sob rótulos superiores (spanner labels) e categorizar linhas;
- **Formatação poderosa:** recursos robustos para estilizar e formatar os valores internos das células de maneira atraente.

O propósito do Great Tables é simplificar e elevar o padrão de exibição de dados tabulares em Python, superando as limitações de formatação visual das ferramentas tradicionais. O projeto resolve um problema prático: transformar dados brutos em relatórios e apresentações visualmente profissionais e fáceis de digerir. Em vez de lidar com códigos complexos e estilizações manuais exaustivas, o Great Tables fornece uma API intuitiva que separa a estrutura dos dados de sua apresentação final, garantindo tabelas elegantes com o mínimo de esforço.

2. Métricas Antes da Refatoração

2.1 Radon Antes da Refatoração

2.1.1 Complexidade Ciclomática por Arquivo Antes da Refatoração

Tabela de piores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
great_tables/_options.py	14	5.36	16	75	C	opt_table_font
great_tables/_boxhead.py	4	5.5	8	22	B	cols_label
great_tables/_spanners.py	20	7.05	27	141	D	tab_spanner
great_tables/_scss.py	3	8.67	18	26	C	compile_scss
great_tables/_utils_nanoplots.py	30	9.17	148	275	F	_generate_nanoplot

Tabela de melhores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
great_tables/_heading.py	1	1.0	1	1	A	tab_header
great_tables/_stubhead.py	1	1.0	1	1	A	tab_stubhead
great_tables/quarto.py	1	1.0	1	1	A	is_quarto_render
great_tables/_source_notes.py	1	1.0	1	1	A	tab_source_note
great_tables/_pipe.py	1	1.0	1	1	A	pipe

2.1.2 Complexidade Ciclomática por Função Antes da Refatoração

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
great_tables/_utils_nanoplots.py	function	_generate_nanoplot	F	148
great_tables/_utils_render_html.py	function	create_columns_component_h	F	78
great_tables/_utils_render_html.py	function	_create_row_component_h	F	70
great_tables/_utils_render_html.py	function	create_body_component_h	F	53
great_tables/_spanners.py	function	tab_spanner	D	27

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
great_tables/_data_color/palettes.py	method	lookup	A	1
great_tables/_data_color/palettes.py	method	_interpolate	A	1
great_tables/_data_color/base.py	function	_get_wcag_contrast_ratio	A	1
great_tables/_data_color/base.py	function	_color_name_list	A	1
great_tables/_data_color/base.py	function	_is_short_hex	A	1

2.1.3 Índice de Manutenibilidade Antes da Refatoração

Tabela de piores métricas

arquivo	mi	rank_mi
great_tables/_spanners.py	6.1282	C
great_tables/_utils_nanoplots.py	5.0132	C
great_tables/_gt_data.py	2.7879	C
great_tables/_formats.py	0	C
great_tables/_utils_render_html.py	0	C

Tabela de melhores métricas

arquivo	mi	rank_mi
great_tables/_heading.py	100	A
great_tables/_stubhead.py	100	A
great_tables/_types.py	100	A
great_tables/loc.py	100	A
great_tables/vals.py	100	A

2.1.4 Métricas brutas Antes da Refatoração

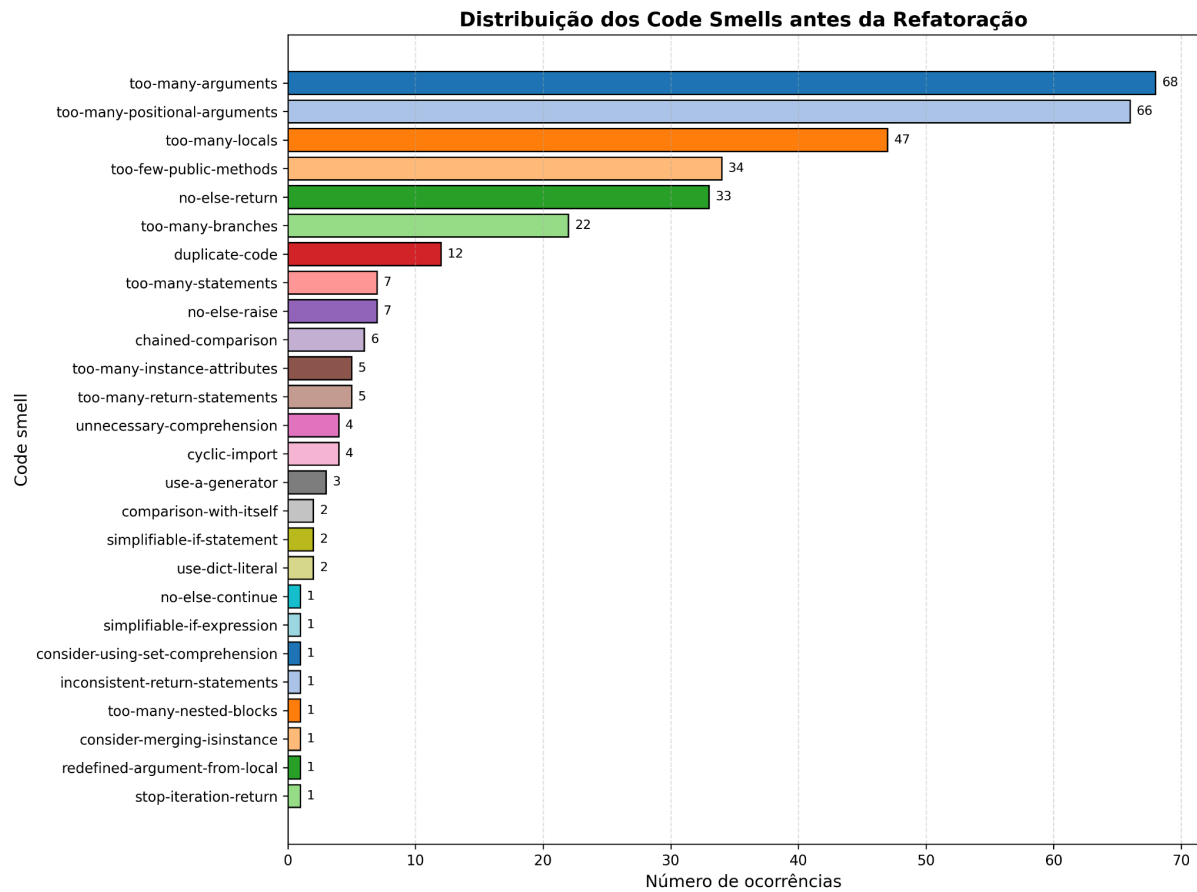
loc total	lloc total	sloc total	comments total	multi
31554	8915	14751	2036	9146

2.2 CodeCarbon Antes da Refatoração

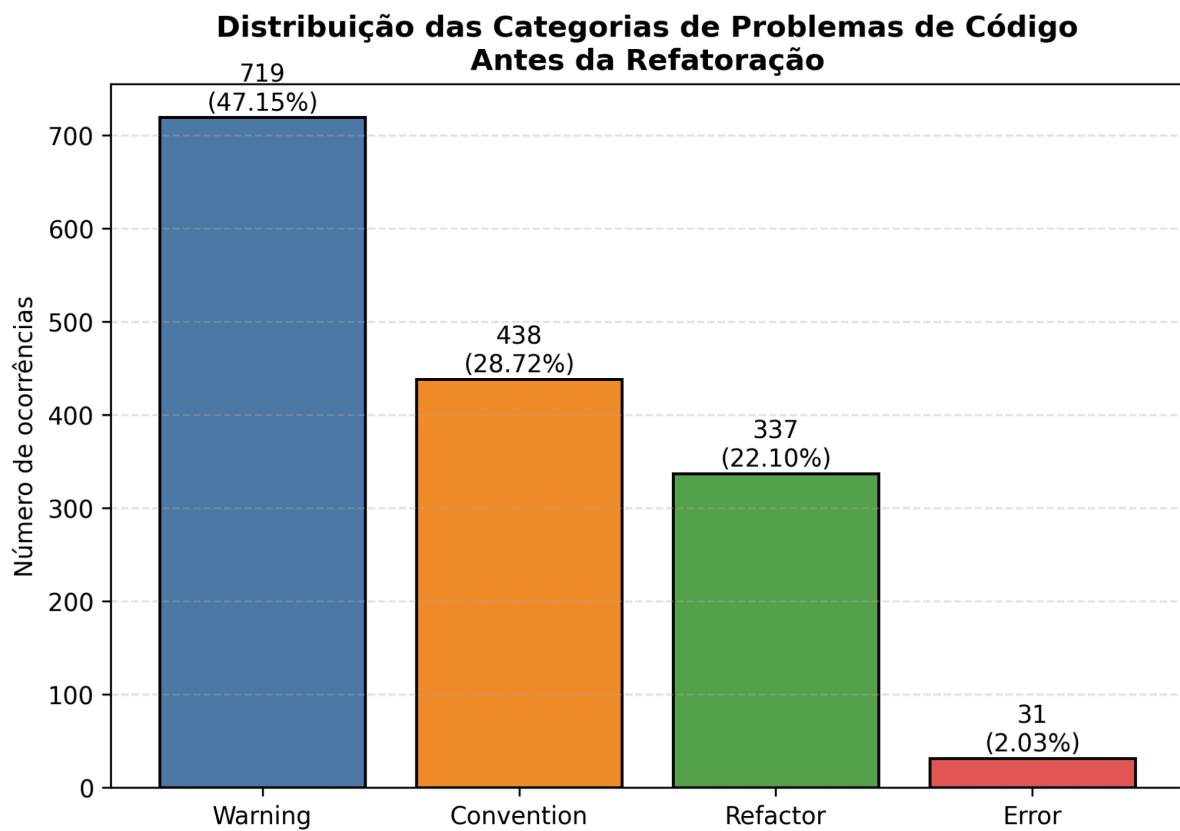
duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
1.554908916	3.89E-07	2.50E-07	1.03E-06	2.92E-06	3.95E-06

2.3 Pylint Antes da Refatoração

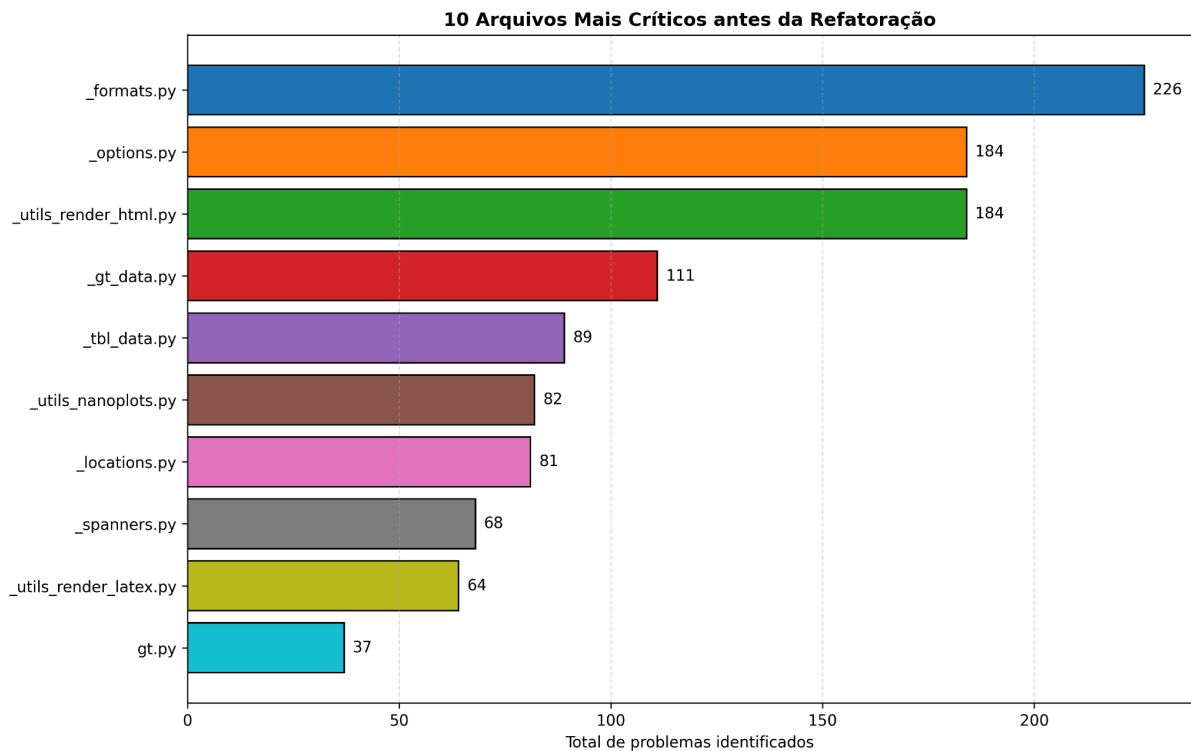
2.3.1 Distribuição do Smells Antes da Refatoração



2.3.2 Distribuição dos Problemas de Código Antes da Refatoração



2.3.3 Arquivos mais Críticos Antes da Refatoração



2.3.4 Score Projeto Antes da Refatoração

Your code has been rated at

7.76/10

2.4 Pytest Antes da Refatoração

2.4.1 Cobertura de testes Antes da Refatoração

<i>File</i>	<i>Statements</i>	<i>Missing</i>	<i>Excluded</i>	<i>Coverage</i>
<i>great_tables/_init_.py</i>	16	2	0	87.5
<i>great_tables/_body.py</i>	5	0	0	100
<i>great_tables/_boxhead.py</i>	63	1	0	98.41269841
<i>great_tables/_cols_merge.py</i>	154	7	0	95.45454545

<i>File</i>	<i>Statements</i>	<i>Missing</i>	<i>Excluded</i>	<i>Coverage</i>
<i>great_tables/_databackend.py</i>	34	4	0	88.23529412
<i>great_tables/_export.py</i>	140	77	0	45
<i>great_tables/_footnotes.py</i>	24	4	0	83.33333333
<i>great_tables/_formats.py</i>	1396	107	0	92.33524355
<i>great_tables/_formats_vals.py</i>	116	24	0	79.31034483
<i>great_tables/_gt_data.py</i>	719	26	0	96.38386648
<i>great_tables/_heading.py</i>	6	0	0	100
<i>great_tables/_helpers.py</i>	246	5	0	97.96747967
<i>great_tables/_locale.py</i>	124	3	0	97.58064516
<i>great_tables/_locations.py</i>	374	12	0	96.79144385
<i>great_tables/_modify_rows.py</i>	88	0	0	100
<i>great_tables/_options.py</i>	173	7	0	95.95375723
<i>great_tables/_pipe.py</i>	6	0	0	100
<i>great_tables/_render.py</i>	25	3	0	88
<i>great_tables/_render_checks.py</i>	15	0	0	100
<i>great_tables/_row_groups.py</i>	0	0	0	0
<i>great_tables/_scss.py</i>	71	4	0	94.36619718
<i>great_tables/_source_notes.py</i>	5	0	0	100
<i>great_tables/_spanners.py</i>	296	19	0	93.58108108
<i>great_tables/_stub.py</i>	4	0	0	100
<i>great_tables/_stubhead.py</i>	5	0	0	100
<i>great_tables/_styles.py</i>	126	11	0	91.26984127
<i>great_tables/_substitution.py</i>	132	5	0	96.21212121

File	Statements	Missing	Excluded	Coverage
<i>great_tables/_tab_create_modify.py</i>	69	1	0	98.55072464
<i>great_tables/_tbl_data.py</i>	467	44	0	90.57815846
<i>great_tables/_tbl_data_align.py</i>	46	6	0	86.95652174
<i>great_tables/_text.py</i>	71	1	0	98.5915493
<i>great_tables/_types.py</i>	2	2	0	0
<i>great_tables/_utils.py</i>	139	3	0	97.84172662
<i>great_tables/_utils_nanoplots.py</i>	637	70	0	89.01098901
<i>great_tables/_utils_render_html.py</i>	617	31	0	94.97568882
<i>great_tables/_utils_render_latex.py</i>	295	3	0	98.98305085
<i>great_tables/_utils_selenium.py</i>	54	14	0	74.07407407
<i>great_tables/gt.py</i>	253	5	0	98.02371542
<i>great_tables/loc.py</i>	19	0	0	100
<i>great_tables/quarto.py</i>	3	0	0	100
<i>great_tables/shiny.py</i>	22	10	0	54.54545455
<i>great_tables/style.py</i>	3	0	0	100
<i>great_tables/utils_render_common.py</i>	17	3	0	82.35294118
<i>great_tables/vals.py</i>	2	0	0	100
<i>great_tables/_data_color/__init__.py</i>	2	0	0	100
<i>great_tables/_data_color/base.py</i>	177	4	0	97.74011299
<i>great_tables/_data_color/constants.py</i>	6	0	0	100
<i>great_tables/_data_color/palettes.py</i>	85	5	0	94.11764706
<i>great_tables/data/__init__.py</i>	115	8	0	93.04347826
<i>tests/__init__.py</i>	0	0	0	0

<i>File</i>	<i>Statements</i>	<i>Missing</i>	<i>Excluded</i>	<i>Coverage</i>
<i>tests/conftest.py</i>	39	0	0	100
<i>tests/test__boxhead.py</i>	99	0	0	100
<i>tests/test__formats_vals.py</i>	9	0	0	100
<i>tests/test__render_checks.py</i>	30	1	0	96.66666667
<i>tests/test__stubhead.py</i>	11	0	0	100
<i>tests/test__utils_nanoplots.py</i>	398	0	0	100
<i>tests/test__utils_selenium.py</i>	14	1	0	92.85714286
<i>tests/test_body.py</i>	6	0	0	100
<i>tests/test_boxhead.py</i>	83	0	0	100
<i>tests/test_cols_merge.py</i>	498	0	0	100
<i>tests/test_datasets.py</i>	44	0	0	100
<i>tests/test_dependencies.py</i>	15	9	0	40
<i>tests/test_export.py</i>	70	0	0	100
<i>tests/test_fmt_duration.py</i>	362	0	0	100
<i>tests/test_fmt_partsper.py</i>	144	0	0	100
<i>tests/test_footnotes.py</i>	573	0	0	100
<i>tests/test_formats.py</i>	651	3	0	99.53917051
<i>tests/test_formats_nanoplots.py</i>	39	0	0	100
<i>tests/test_formats_vals.py</i>	32	0	0	100
<i>tests/test_gt.py</i>	30	0	0	100
<i>tests/test_gt_data.py</i>	29	0	0	100
<i>tests/test_gtsave.py</i>	75	51	0	32
<i>tests/test_gtsave_validation.py</i>	26	4	0	84.61538462

File	Statements	Missing	Excluded	Coverage
<i>tests/test_heading.py</i>	9	0	0	100
<i>tests/test_helpers.py</i>	143	14	0	90.20979021
<i>tests/test_locations.py</i>	158	1	0	99.36708861
<i>tests/test_modify_rows.py</i>	212	0	0	100
<i>tests/test_nanoplots_snap.py</i>	64	0	0	100
<i>tests/test_options.py</i>	390	0	0	100
<i>tests/test_pipe.py</i>	14	0	0	100
<i>tests/test_repr.py</i>	31	0	0	100
<i>tests/test_row_groups.py</i>	35	0	0	100
<i>tests/test_scss.py</i>	18	0	0	100
<i>tests/test_shiny.py</i>	1	0	0	100
<i>tests/test_spanners.py</i>	289	0	0	100
<i>tests/test_styles.py</i>	63	0	0	100
<i>tests/test_substitutions.py</i>	317	5	0	98.42271293
<i>tests/test_summary_rows_snap.py</i>	201	0	0	100
<i>tests/test_tab_create_modify.py</i>	219	0	0	100
<i>tests/test_tbl_data.py</i>	257	3	0	98.83268482
<i>tests/test_tbl_data_align.py</i>	129	0	0	100
<i>tests/test_text.py</i>	64	0	0	100
<i>tests/test_utils.py</i>	105	0	0	100
<i>tests/test_utils_render_common.py</i>	13	0	0	100
<i>tests/test_utils_render_html.py</i>	98	3	0	96.93877551
<i>tests/test_utils_render_latex.py</i>	250	0	0	100

<i>File</i>	<i>Statements</i>	<i>Missing</i>	<i>Excluded</i>	<i>Coverage</i>
<i>tests/utlis.py</i>	<i>5</i>	<i>0</i>	<i>0</i>	<i>100</i>
<i>tests/data_color/test_data_color.py</i>	<i>136</i>	<i>1</i>	<i>0</i>	<i>99.26470588</i>
<i>tests/data_color/test_data_color_utlis.py</i>	<i>121</i>	<i>0</i>	<i>0</i>	<i>100</i>

2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração

Quantidade de testes que passaram: 2086

Quantidade de testes que falharam: 3

3. Processo de Refatoração

Modelo utilizado: ChatGPT 5.4 Mini

3.1 too-many-statements

Quantidade de ocorrências: 7

3.1.1 Ocorrência Número 1

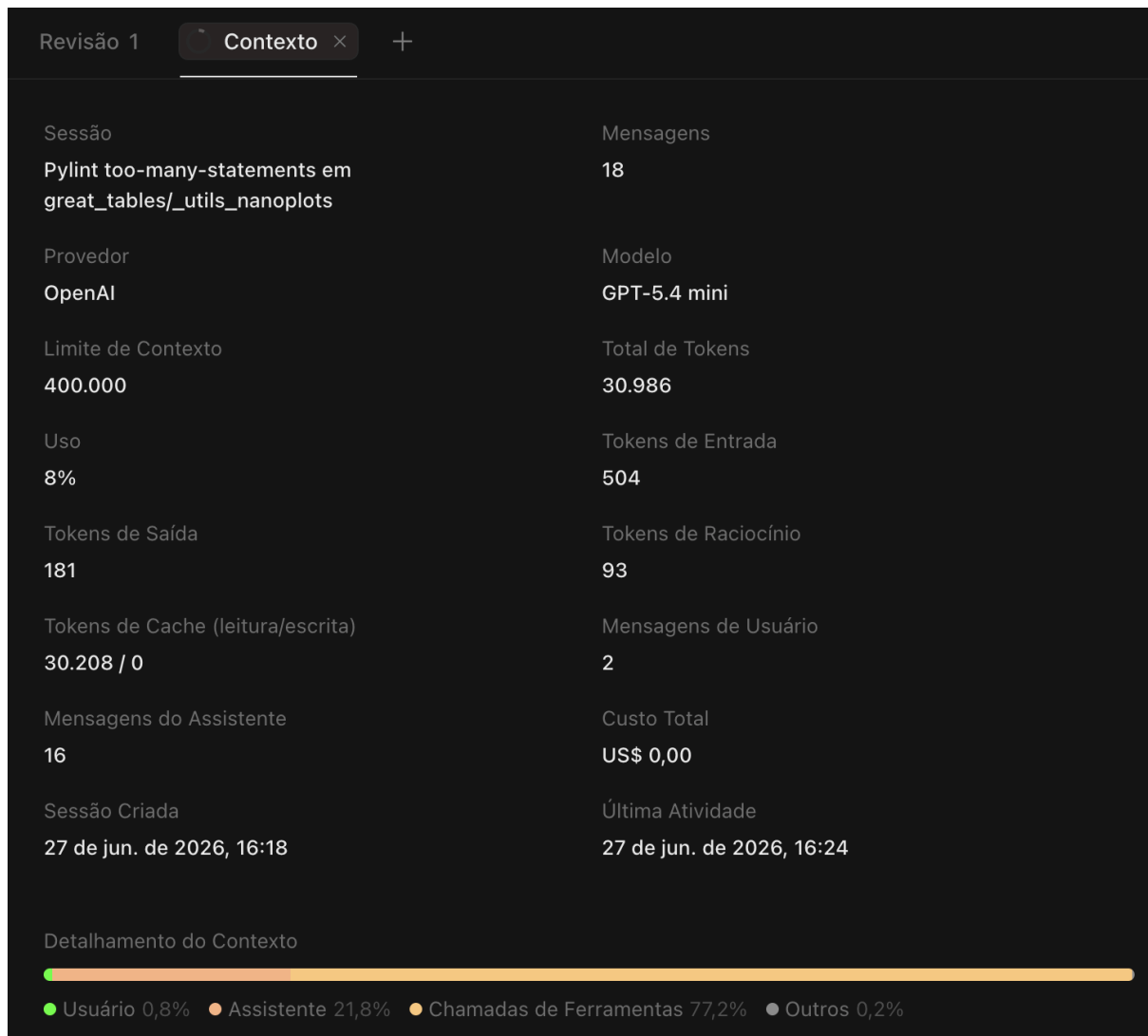
Tabela de identificação

module	obj	message
great_tables._utlis_nanoplots	_format_number_compactly	Too many statements (65/50)

Seguiu o plano de refatoração: Sim, o plano de refatoração foi seguido de forma consistente, pois a função `_format_number_compactly` teve suas responsabilidades distribuídas em helpers específicos para tratamento de formatação personalizada, definição do perfil numérico e aplicação da formatação final. Essa decomposição está alinhada com a técnica recomendada para corrigir o smell R0915 (too-many-statements), reduzindo a quantidade de instruções na função principal sem alterar sua responsabilidade como orquestradora do fluxo. Além disso, a refatoração buscou preservar o comportamento existente, mantendo a lógica de tratamento de valores especiais, escalas numéricas, moedas, inteiros e notação científica, não havendo indícios de introdução de novos smells críticos.

Link da publicação OpenCode: <https://opncd.ai/share/1HXbxMu0>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função concentrava diversas regras de negócio relacionadas à formatação numérica, tornando o código extenso e de difícil manutenção. A extração de helpers especializados aumentou a modularidade, facilitou o entendimento da lógica e tornou futuras alterações mais localizadas e menos propensas a introduzir regressões. Embora a validação tenha ficado limitada à compilação do arquivo devido à indisponibilidade das dependências necessárias para execução dos testes automatizados, a estratégia adotada é coerente com boas práticas de refatoração e atende diretamente ao objetivo de reduzir a complexidade estrutural da função. Nesse contexto, o uso da LLM mostrou-se adequado, pois automatizou uma reorganização relativamente extensa do código, produzindo um ganho de legibilidade e manutenibilidade que compensou o esforço empregado.

3.1.2 Ocorrência Número 2

Tabela de identificação

module	obj	message
--------	-----	---------

great_tables._utils_nanoplots	_generate_nanoplot	Too many statements (410/50)
-------------------------------	--------------------	------------------------------

Seguiu o plano de refatoração: Sim, a refatoração seguiu efetivamente o plano proposto ao extrair responsabilidades específicas da implementação principal para helpers dedicados à geração de linhas, pontos e barras verticais. Essa estratégia está alinhada com a técnica recomendada para corrigir o smell R0915 (too-many-statements), pois reduz gradativamente a quantidade de instruções concentradas na função principal, transformando-a em um orquestrador responsável apenas por coordenar a execução das etapas. A preservação da API pública e a preocupação em manter o comportamento existente durante a migração indicam que a refatoração foi conduzida de forma incremental, sem evidências de introdução de novos smells críticos.

Link da publicação OpenCode: <https://opncd.ai/share/FfSojiGR>

Print consumo de token:

Revisão 1	Contexto x +
Sessão	Mensagens
Pylint R0915 em _generate_nanoplot	41
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	85.995
Uso	Tokens de Entrada
21%	1.098
Tokens de Saída	Tokens de Raciocínio
167	762
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
83.968 / 0	4
Mensagens do Assistente	Custo Total
37	US\$ 0,00
Sessão Criada	Última Atividade
27 de jun. de 2026, 21:43	27 de jun. de 2026, 22:03
Detalhamento do Contexto	
● Usuário 0,3% ● Assistente 30,4% ● Chamadas de Ferramentas 69,2% ● Outros 0,1%	

Análise da refatoração: A refatoração valeu a pena porque promoveu uma redução real da complexidade estrutural da função, isolando partes independentes da lógica em helpers com responsabilidades bem definidas. Essa decomposição melhora a legibilidade, facilita a manutenção e torna cada componente mais simples de compreender e testar individualmente, além de preparar o código para futuras evoluções sem concentrar novas responsabilidades em uma única função. Apesar de a validação ter se restringido à compilação do código devido à indisponibilidade do ambiente de

testes, a estratégia incremental adotada reduz o risco de regressões e permite verificar cada etapa da refatoração de forma controlada. Nesse contexto, o uso da LLM mostrou-se apropriado, pois automatizou uma reorganização significativa do código, gerando benefícios de manutenção que justificam o esforço empregado, ainda que a decomposição completa da função permaneça em andamento.

3.1.3 Ocorrência Número 3

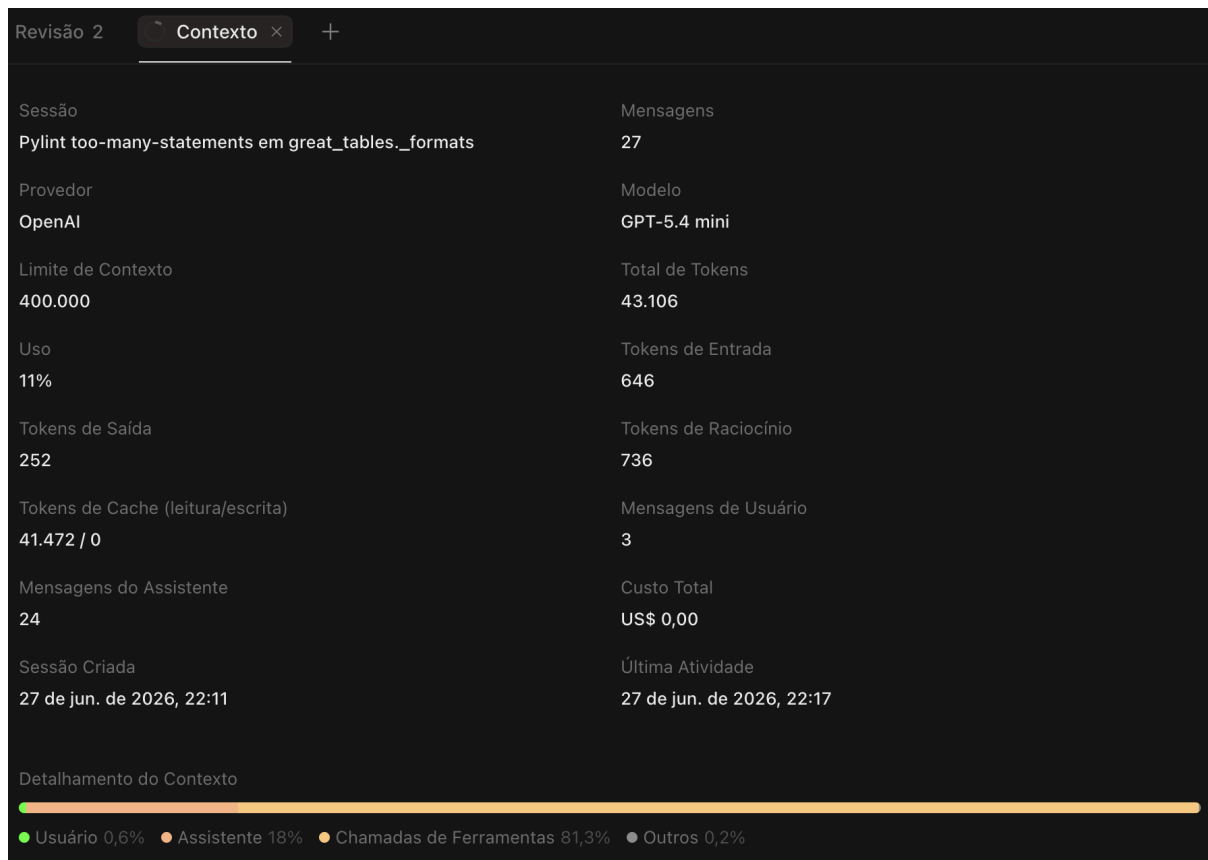
Tabela de identificação

module	obj	message
great_tables._formats	fmt_nanoplot	Too many statements (60/50)

Seguiu o plano de refatoração: Sim, o plano de refatoração foi seguido integralmente, uma vez que a função `fmt_nanoplot` teve suas múltiplas responsabilidades distribuídas em helpers privados especializados, incluindo validação de argumentos, detecção de colunas escalares, coleta de valores, cálculo de autoscale, resolução de opções e formatação das células. Essa abordagem está alinhada com a técnica recomendada para corrigir o smell R0915 (too-many-statements), transformando a função principal em um orquestrador enxuto sem alterar sua assinatura pública nem a API do módulo. O resultado informado, com a redução de 60 para apenas 9 statements, demonstra que o objetivo da refatoração foi alcançado, sem indícios de introdução de novos smells críticos ou de alterações no comportamento esperado do sistema.

Link da publicação OpenCode: <https://opncd.ai/share/RVpv8Xrv>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função original concentrava diversas responsabilidades relacionadas à validação, preparação de dados, configuração de parâmetros e geração dos nanoplots, tornando o código extenso e de difícil manutenção. A extração dessas responsabilidades para helpers especializados aumentou significativamente a modularidade, favorecendo a legibilidade, a reutilização de código e a testabilidade de cada etapa do processamento de forma independente. Além disso, a expressiva redução da quantidade de statements evidencia que o smell foi efetivamente eliminado, sem comprometer a interface pública da função. Embora a validação tenha se limitado à compilação do arquivo devido à indisponibilidade das ferramentas de teste e lint no ambiente utilizado, a estratégia adotada é consistente com as boas práticas de refatoração e apresenta baixo risco de regressão. Nesse cenário, o uso da LLM mostrou-se bastante eficiente, pois automatizou uma reorganização estrutural relativamente extensa do código, produzindo ganhos claros de qualidade e manutenção que justificam plenamente o esforço e o custo envolvidos.

3.1.4 Ocorrência Número 4

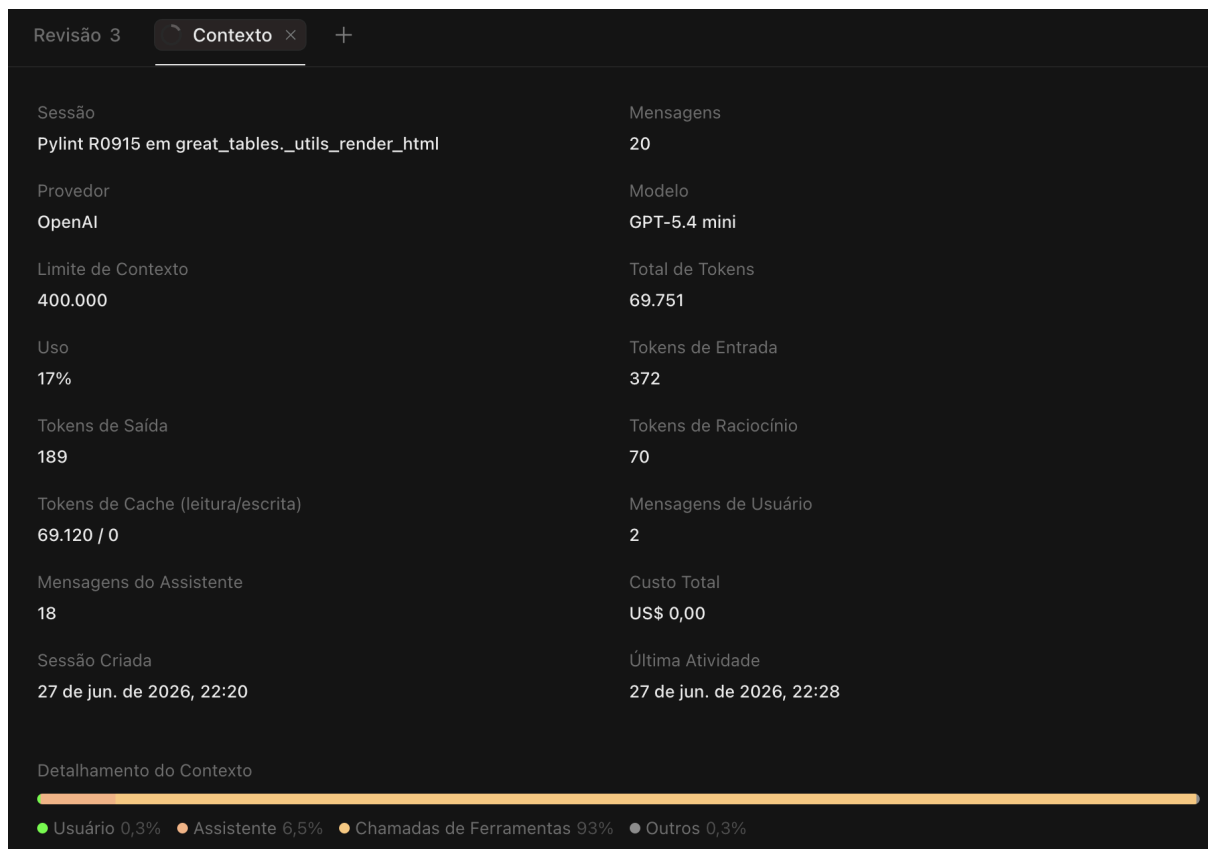
Tabela de identificação

module	obj	message
great_tables._utils_render_html	create_columns_component_h	Too many statements (90/50)

Seguiu o plano de refatoração: Sim, o plano de refatoração foi seguido de maneira consistente, pois a função `create_columns_component_h` teve sua lógica de construção do cabeçalho HTML distribuída em helpers especializados para o processamento do stubhead, dos cabeçalhos de coluna, dos diferentes níveis de spanners e dos casos sem spanners. Essa decomposição está alinhada com a técnica recomendada para corrigir o smell R0915 (too-many-statements), reduzindo significativamente a quantidade de instruções concentradas na função principal e transformando-a em um orquestrador responsável apenas pelo fluxo de execução. A estratégia também preservou a responsabilidade de cada etapa da renderização e não há indícios de introdução de novos smells críticos ou de alterações na interface da função.

Link da publicação OpenCode: <https://opncd.ai/share/pPudcusp>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função original reunia diversas responsabilidades relacionadas à montagem do cabeçalho HTML, contemplando diferentes cenários de renderização e repetindo parte da lógica para tratamento de spanners, estilos e notas de rodapé. A extração desses blocos para helpers especializados tornou o código mais modular, legível e fácil de manter, além de favorecer a criação de testes unitários direcionados para cada parte da geração do HTML. Outro benefício importante foi deixar explícita a separação entre a coordenação do fluxo e a

implementação dos detalhes da renderização, o que reduz o risco de regressões em futuras alterações. Embora a validação tenha ficado restrita à compilação do arquivo em razão da indisponibilidade das dependências necessárias para execução dos testes automatizados, a estratégia empregada é compatível com boas práticas de refatoração e atende diretamente ao objetivo de reduzir a complexidade da função. Nesse contexto, o uso da LLM mostrou-se eficiente ao automatizar uma reorganização estrutural extensa, produzindo ganhos de legibilidade e manutenção que justificam o esforço realizado.

3.1.5 Ocorrência Número 5

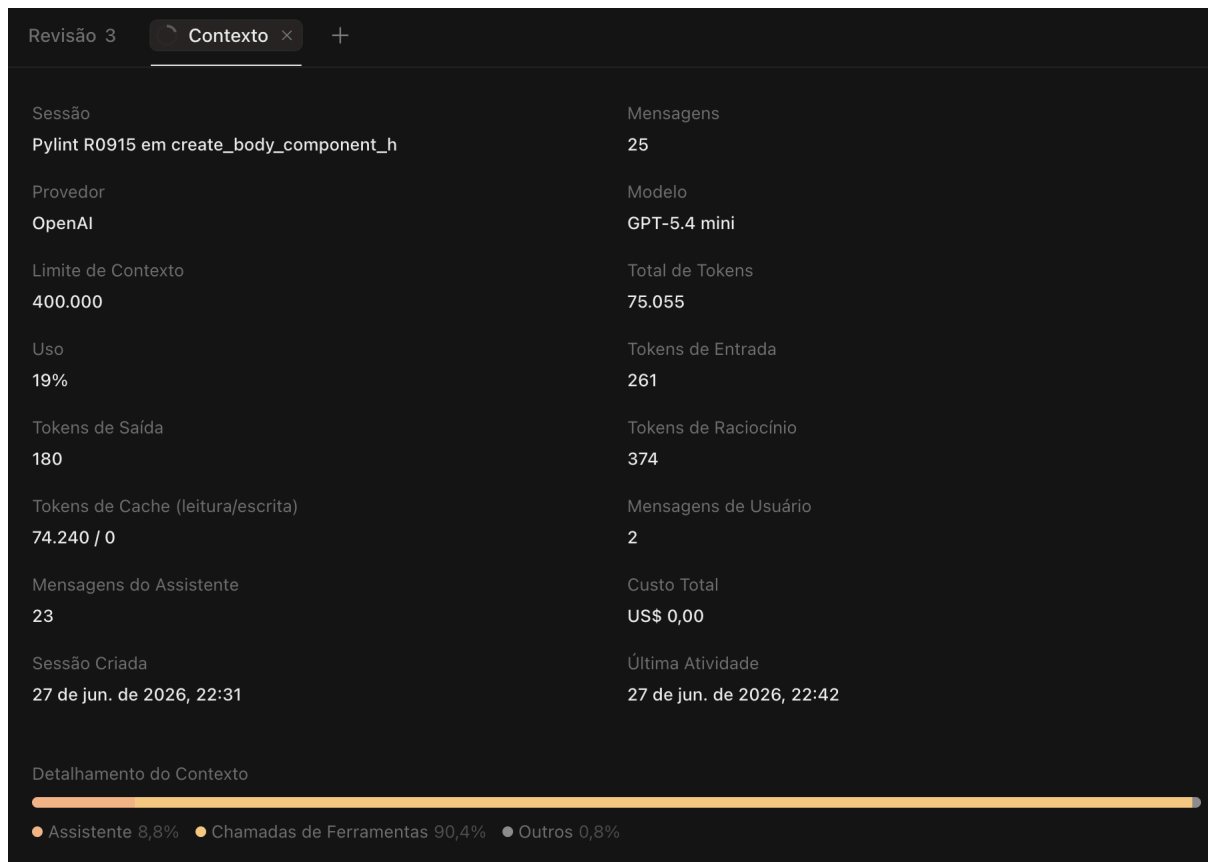
Tabela de identificação

module	obj	message
great_tables._utils_render_html	create_body_component_h	Too many statements (70/50)

Seguiu o plano de refatoração: Sim, o plano de refatoração foi seguido de forma consistente, pois a função `create_body_component_h` deixou de concentrar toda a lógica de preparação e renderização do corpo da tabela, passando a atuar apenas como orquestradora do fluxo. A extração da preparação do contexto para `_prepare_body_component_h`, o agrupamento das informações em `_BodyComponentContextH` e a separação da renderização das linhas de resumo e dos grupos em helpers específicos estão alinhados com a técnica recomendada para eliminar o smell R0915 (too-many-statements). A redução estimada de 70 para 7 statements demonstra que a complexidade da função principal foi efetivamente reduzida, sem indícios de alterações na interface pública ou da introdução de novos smells críticos.

Link da publicação OpenCode: <https://opncd.ai/share/cnCwn6Ss>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função original acumulava responsabilidades relacionadas à preparação dos dados, organização do estado interno e renderização das diferentes partes do corpo da tabela, o que dificultava sua compreensão e manutenção. A introdução de um objeto de contexto para encapsular as informações compartilhadas entre os helpers também contribuiu para evitar a criação de funções com listas extensas de parâmetros, tornando a solução mais coesa e de mais fácil evolução. Além de reduzir significativamente a complexidade estrutural da função principal, a nova organização favorece a reutilização de código e a criação de testes unitários para cada etapa da renderização. Embora a validação tenha se limitado à compilação do módulo em razão da indisponibilidade do ambiente de testes, a estratégia adotada é consistente com boas práticas de refatoração e apresenta baixo risco de regressão funcional. Nesse contexto, o uso da LLM mostrou-se bastante adequado, pois automatizou uma reorganização estrutural extensa do código, produzindo ganhos claros de legibilidade, modularidade e manutenibilidade que justificam o esforço empregado.

3.1.6 Ocorrência Número 6

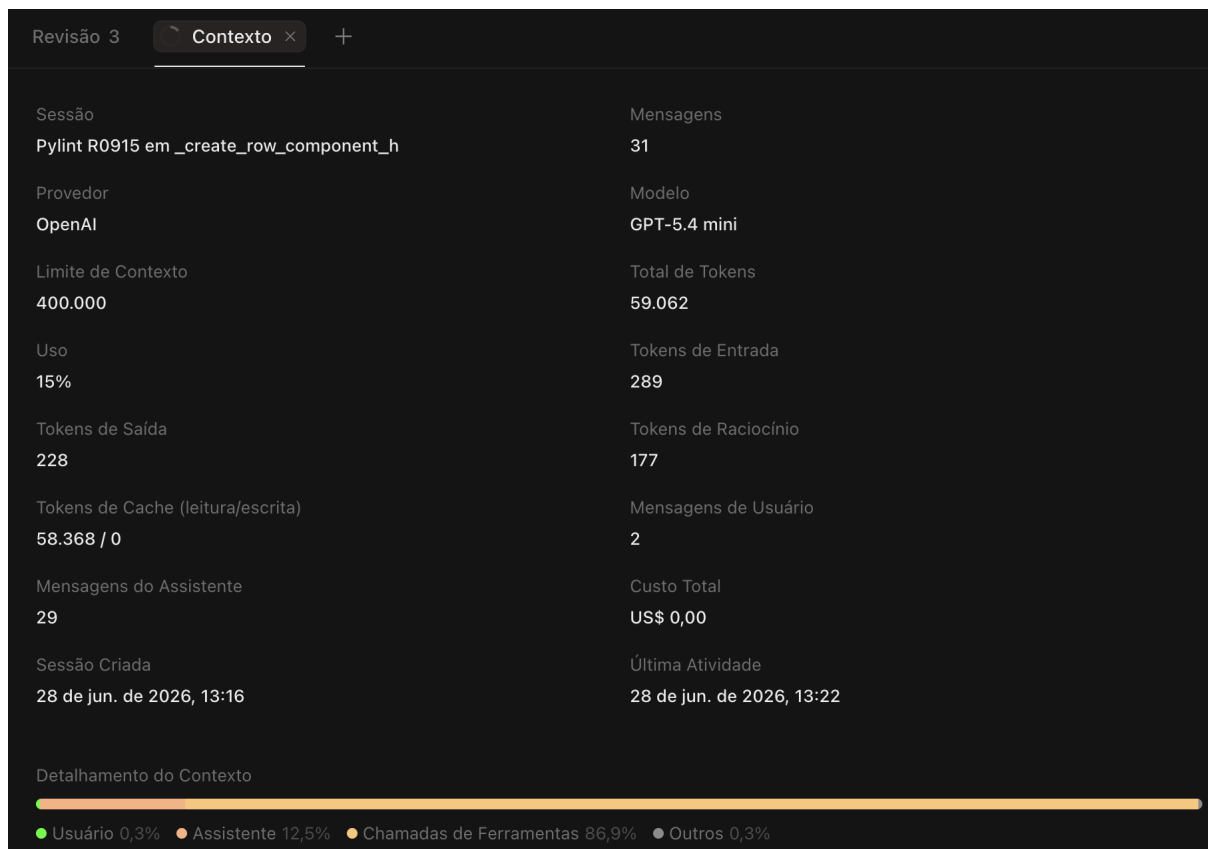
Tabela de identificação

module	obj	message
great_tables._utils_render_html	_create_row_component_h	Too many statements (85/50)

Seguiu o plano de refatoração: Sim, o plano de refatoração foi seguido de forma consistente, pois a função `_create_row_component_h` teve suas responsabilidades distribuídas em helpers especializados para montagem da célula de summary, resolução do conteúdo da célula, seleção de footnotes e geração do HTML final. Essa decomposição está alinhada com a técnica recomendada para corrigir o smell R0915 (too-many-statements), reduzindo significativamente a quantidade de instruções concentradas na função principal e transformando-a em um orquestrador responsável apenas pelo fluxo de criação das linhas. A redução da contagem para 17 statements confirma que o objetivo da refatoração foi atingido, sem indícios de alteração da interface da função ou da introdução de novos smells críticos.

Link da publicação OpenCode: <https://opncd.ai/share/VUN2z7yk>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função original concentrava diferentes responsabilidades relacionadas à renderização das linhas da tabela, incluindo tratamento de summary rows, definição do conteúdo das células, aplicação de footnotes, cálculo de classes CSS e geração do HTML. Ao isolar essas responsabilidades em helpers específicos, o código tornou-se mais modular, legível e de mais fácil manutenção, além de permitir que cada etapa da renderização seja testada de forma independente. Essa organização também reduz o risco de regressões, pois futuras

modificações em uma das responsabilidades tendem a impactar apenas o helper correspondente, preservando o restante do fluxo. Embora a validação tenha ficado restrita à compilação do módulo e à verificação da contagem de statements via AST, devido à ausência das dependências necessárias para executar os testes automatizados, as evidências apresentadas indicam que a refatoração atingiu seu objetivo de eliminar o smell sem comprometer a estrutura da renderização. Nesse contexto, o uso da LLM mostrou-se bastante eficaz, automatizando uma reorganização estrutural significativa e produzindo ganhos claros de legibilidade, modularidade e manutenibilidade que justificam o esforço empregado.

3.1.7 Ocorrência Número 7

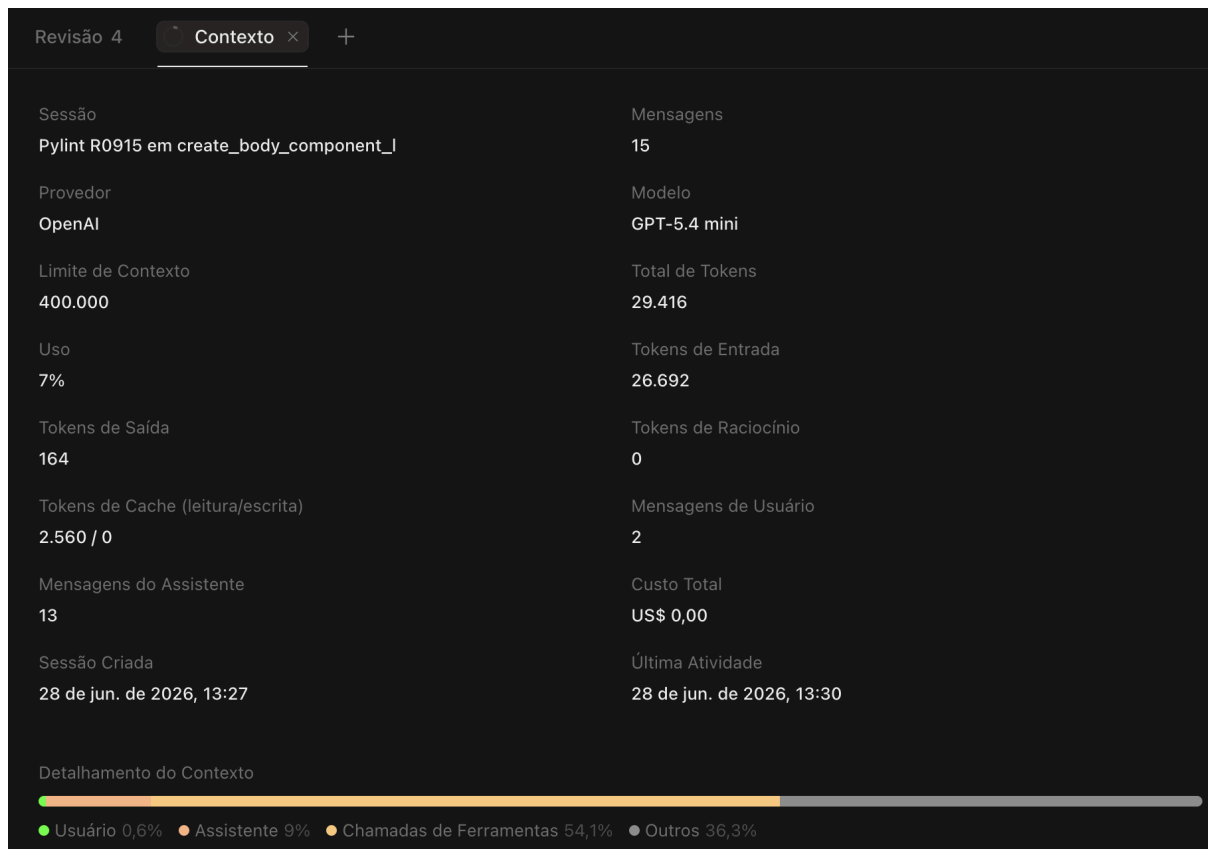
Tabela de identificação

module	obj	message
great_tables._utils_render_latex	create_body_component_l	Too many statements (67/50)

Seguiu o plano de refatoração: Sim, o plano de refatoração foi seguido de forma adequada, pois a função `create_body_component_l` teve suas responsabilidades distribuídas em helpers específicos para montagem de linhas, inserção de cabeçalhos de grupo e geração dos diferentes tipos de summary rows. Essa estratégia está alinhada com a técnica recomendada para corrigir o smell R0915 (too-many-statements), reduzindo a quantidade de instruções concentradas na função principal e transformando-a em um orquestrador responsável apenas pela coordenação do fluxo de geração do corpo da tabela em LaTeX. A preocupação em preservar as condições de execução dos group summaries durante a refatoração demonstra cuidado com a manutenção do comportamento original, sem indícios da introdução de novos smells críticos.

Link da publicação OpenCode: <https://opnecd.ai/share/ORTmCilq>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função original concentrava diversas responsabilidades relacionadas à construção do corpo da tabela em LaTeX, incluindo preparação do layout, renderização das linhas de dados, tratamento de grupos e inserção de linhas de resumo. A extração dessas funcionalidades para helpers especializados tornou a implementação mais modular, legível e fácil de manter, além de favorecer a reutilização e a criação de testes unitários mais específicos para cada etapa da renderização. A redução da função principal para 22 statements evidencia que o objetivo de eliminar o smell foi alcançado de forma efetiva, preservando a estrutura da API e a lógica de geração do documento. Embora a validação tenha se restringido à compilação do módulo e à verificação da contagem de statements, devido à ausência do ambiente completo de testes, a estratégia adotada segue boas práticas de refatoração e apresenta baixo risco de regressão. Nesse contexto, o uso da LLM mostrou-se bastante eficaz ao automatizar uma reorganização estrutural significativa do código, proporcionando ganhos claros de legibilidade, modularidade e manutenibilidade que justificam o esforço realizado.

3.2 too-many-instance-attributes

Quantidade de ocorrências: 5

3.2.1 Ocorrência Número 1

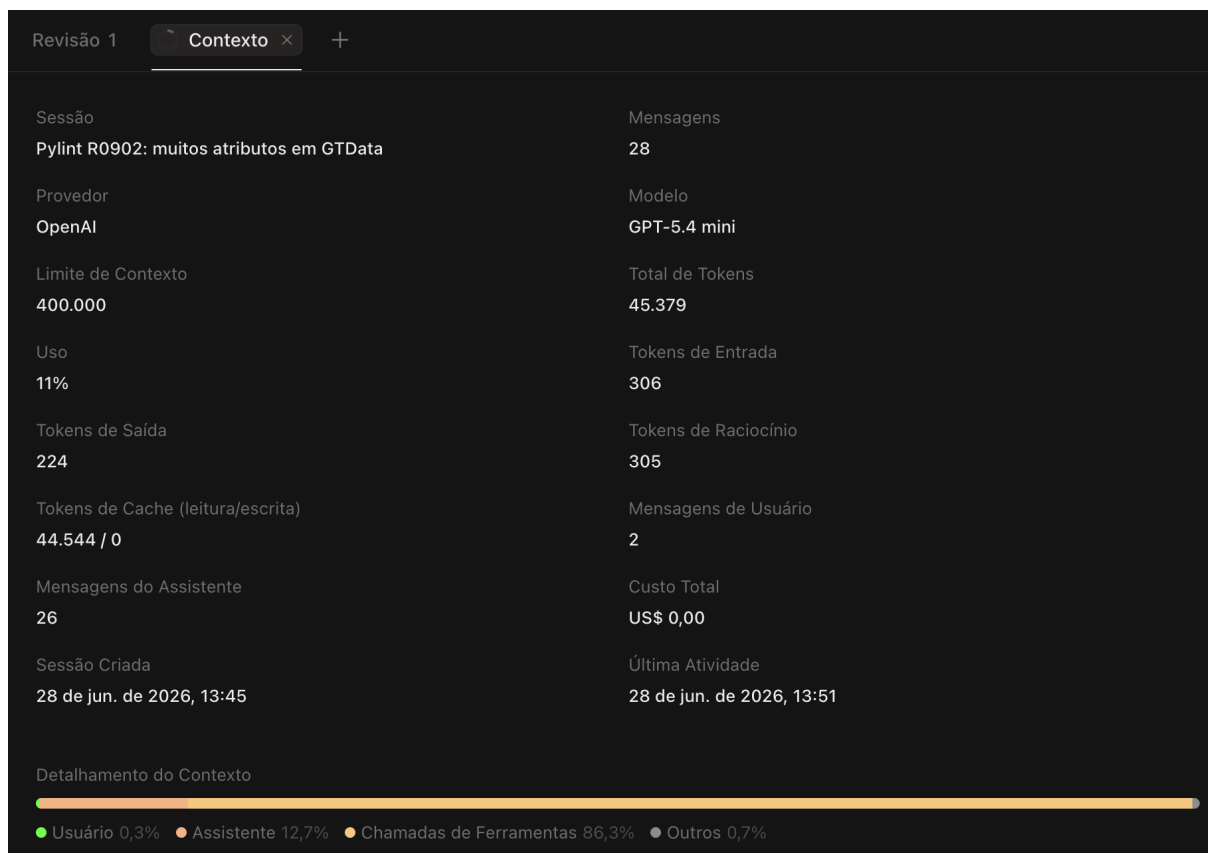
Tabela de identificação

module	obj	message
great_tables._gt_data	GTData	Too many instance attributes (20/7)

Seguiu o plano de refatoração: Sim, o plano de refatoração foi seguido de forma consistente, pois a classe GTData deixou de concentrar todos os seus atributos diretamente na instância, passando a agrupá-los em subestruturas especializadas (GTCoreData, GTSectionData, GTStyleData, GTTransformData e GTSettingsData). Essa abordagem está alinhada com a técnica recomendada para corrigir o smell R0902 (too-many-instance-attributes), reduzindo significativamente a quantidade de atributos da classe principal sem recorrer à simples desativação da regra do Pylint. Além disso, a preservação da API existente por meio de propriedades e a adaptação do método `_replace()` demonstram a preocupação em manter a compatibilidade com o restante do sistema, sem indícios de introdução de novos smells críticos.

Link da publicação OpenCode: <https://opncl.ai/share/4LXvVX7b>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque atacou um problema de projeto mais profundo do que uma simples questão de organização do código. A classe GTData exercia o papel de

um grande agregador de estado, reunindo dados, configurações, elementos de renderização, estilos e transformações em uma única estrutura, o que comprometia sua coesão e dificultava sua evolução. O agrupamento desses atributos em subestruturas temáticas tornou o modelo de dados mais organizado e expressivo, favorecendo a manutenção, a compreensão da arquitetura e a possibilidade de evolução independente de cada conjunto de responsabilidades. A decisão de preservar a interface existente por meio de propriedades minimizou o impacto da mudança sobre o restante da base de código, reduzindo o risco de regressões. Embora a validação tenha sido limitada à compilação do projeto e a testes pontuais de importação e do método `_replace()`, devido à indisponibilidade do ambiente completo de testes, a estratégia adotada representa uma melhoria arquitetural consistente e compatível com boas práticas de orientação a objetos. Nesse contexto, o uso da LLM mostrou-se especialmente vantajoso, pois automatizou uma refatoração estrutural de maior porte e potencial impacto, produzindo ganhos significativos de coesão e organização que justificam plenamente o esforço empregado.

3.2.2 Ocorrência Número 2

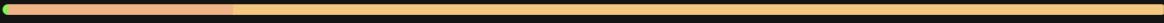
Tabela de identificação

module	obj	message
great_tables._gt_data	Options	Too many instance attributes (146/7)

Seguiu o plano de refatoração: Sim, o plano de refatoração foi seguido de forma consistente, pois a classe `Options` deixou de armazenar diretamente seus 146 atributos de instância e passou a encapsular esse estado em uma única estrutura interna, preservando a interface existente por meio de delegação de atributos. Essa abordagem elimina a causa do smell R0902 (too-many-instance-attributes) sem recorrer à simples desativação da regra do Pylint, mantendo compatibilidade com o restante da aplicação. A adaptação de pontos críticos, como `tab_options()` e `compile_scss()`, demonstra que a refatoração considerou os principais consumidores da classe, preservando o comportamento esperado e não apresentando indícios da introdução de novos smells críticos.

Link da publicação OpenCode: <https://opncd.ai/share/kuNHdmxD>

Print consumo de token:

Revisão 3	Contexto × +
Sessão	Mensagens
Pylint R0902 em great_tables._gt_data.py	36
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	50.720
Uso	Tokens de Entrada
13%	444
Tokens de Saída	Tokens de Raciocínio
171	953
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
49.152 / 0	2
Mensagens do Assistente	Custo Total
34	US\$ 0,00
Sessão Criada	Última Atividade
28 de jun. de 2026, 13:57	28 de jun. de 2026, 14:04
Detalhamento do Contexto 	
● Usuário 0,5% ● Assistente 19,4% ● Chamadas de Ferramentas 80% ● Outros 0,2%	

Análise da refatoração: A refatoração valeu a pena porque atacou diretamente um problema de projeto que comprometia a coesão da classe Options. Em vez de apenas reorganizar atributos ou suprimi-los do relatório do Pylint, a solução adotou um modelo mais flexível, centralizando o armazenamento das opções em uma estrutura interna e mantendo a compatibilidade da API por meio de delegação de acesso aos atributos. Essa estratégia reduz o acoplamento entre a representação interna e os consumidores da classe, facilita a evolução do conjunto de opções e torna futuras alterações menos custosas, já que novos parâmetros podem ser incorporados sem expandir continuamente a quantidade de atributos da instância. A necessidade de adaptar componentes como `tab_options()` e `compile_scss()` era esperada em uma mudança arquitetural desse porte e foi conduzida de forma consistente. Embora a validação tenha se restringido à compilação do projeto e a testes de fumaça devido à indisponibilidade do ambiente completo de testes, as evidências apresentadas indicam que a compatibilidade funcional foi preservada. Nesse contexto, o uso da LLM mostrou-se especialmente adequado, pois automatizou uma refatoração arquitetural complexa, envolvendo múltiplos pontos de integração, produzindo ganhos relevantes de organização, escalabilidade e manutenção que justificam o esforço empregado.

3.2.3 Ocorrência Número 3

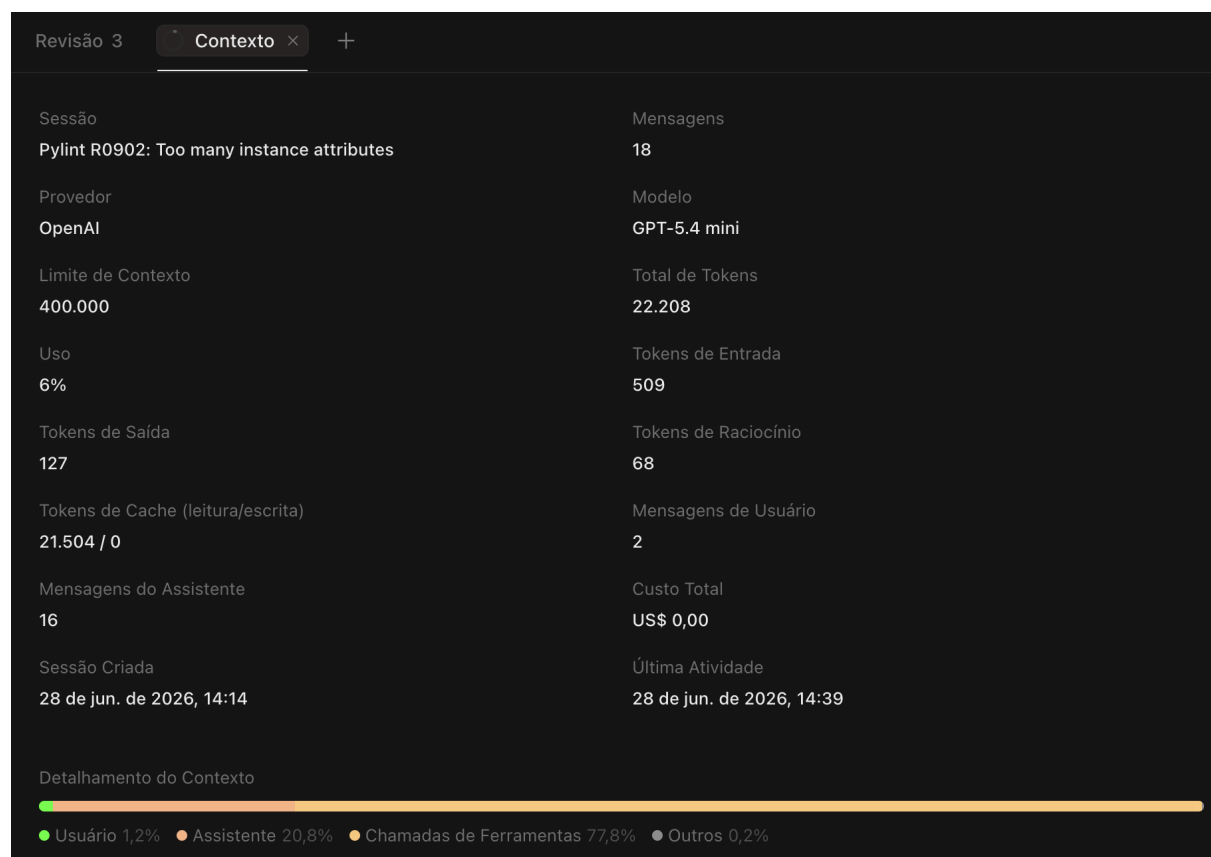
Tabela de identificação

module	obj	message
great_tables._options	StyleMapper	Too many instance attributes (12/7)

Seguiu o plano de refatoração: Sim, o plano de refatoração foi seguido de forma consistente, pois a classe StyleMapper foi eliminada e substituída por uma abordagem baseada em uma estrutura de mapeamento estática (STYLE_MAPPINGS) e uma função pura (map_style_params()). Essa solução está alinhada com a técnica recomendada para corrigir o smell R0902 (too-many-instance-attributes), uma vez que remove completamente a necessidade de manter uma instância com diversos atributos cuja única finalidade era intermediar uma transformação determinística de dados. A atualização de opt_stylize() para utilizar a nova função preserva o comportamento esperado do sistema e não há indícios da introdução de novos smells críticos.

Link da publicação OpenCode: <https://opncd.ai/share/Jxc75moK>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque identificou que a classe não possuía estado ou comportamento que justificassem sua existência como objeto. Ao substituir o @dataclass por uma função pura apoiada em um mapa constante de configurações, o código tornou-se mais simples, explícito e aderente ao propósito da funcionalidade, reduzindo a quantidade de elementos

estruturais desnecessários na base de código. Além de eliminar o alerta do Pylint, essa abordagem melhora a legibilidade, facilita futuras alterações nos mapeamentos e reduz o acoplamento, já que a transformação passa a depender apenas dos dados de entrada. A remoção de imports que se tornaram desnecessários também contribui para a limpeza do módulo. Embora a validação tenha se limitado à compilação do arquivo devido à indisponibilidade do ambiente de testes, a mudança apresenta baixo risco de regressão por se tratar de uma transformação determinística e localizada. Nesse contexto, o uso da LLM mostrou-se bastante eficiente, pois identificou que uma classe inteira poderia ser substituída por uma solução funcional mais simples, produzindo ganhos de clareza e manutenção que justificam plenamente o esforço da refatoração.

3.2.4 Ocorrência Número 4

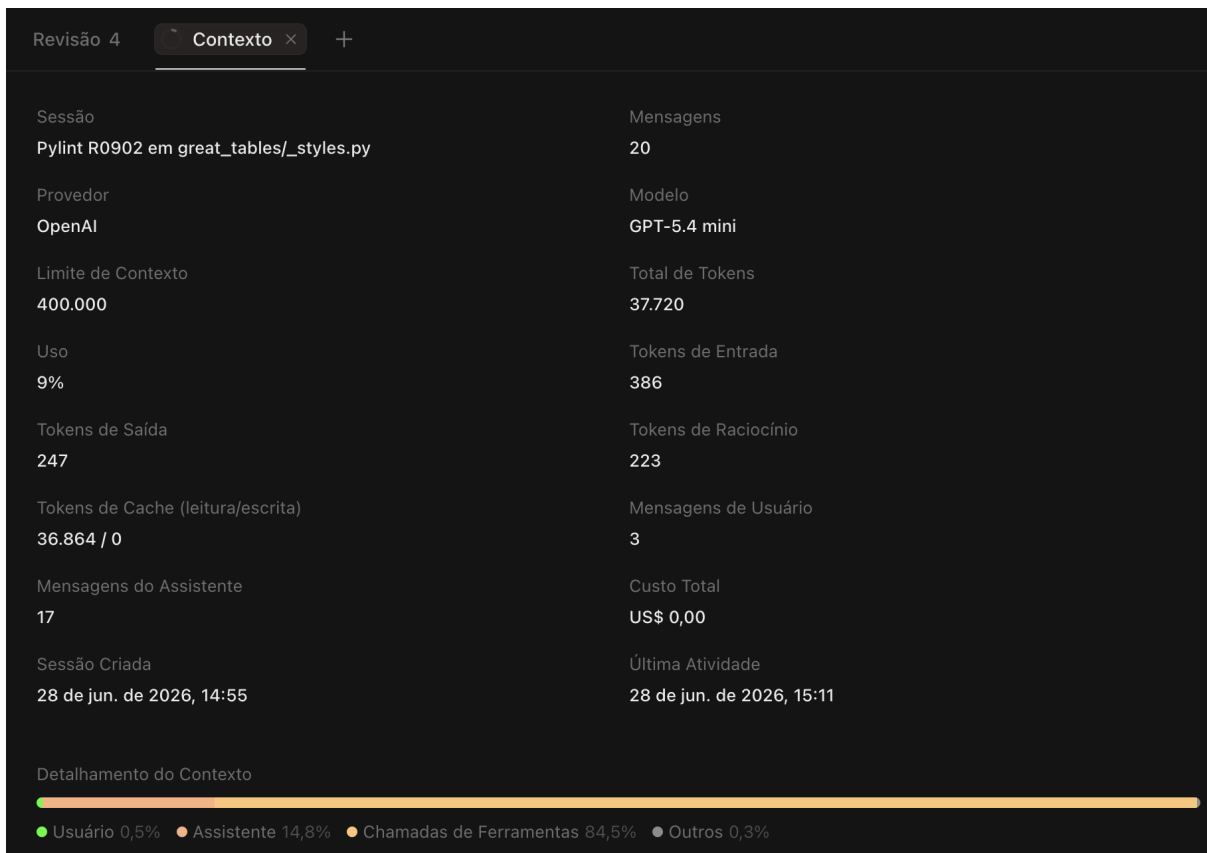
Tabela de identificação

module	obj	message
great_tables._styles	CellStyleText	Too many instance attributes (11/7)

Seguiu o plano de refatoração: Sim, o plano de refatoração foi seguido de forma consistente, pois a classe `CellStyleText` teve seus atributos agrupados em uma estrutura interna (`_CellStyleTextSpec`), reduzindo significativamente a quantidade de atributos diretamente associados à instância e atacando a causa do smell R0902 (too-many-instance-attributes). A estratégia preservou a API pública da classe ao manter a mesma assinatura do construtor e adicionar propriedades delegadas para acesso aos atributos, evitando impactos no restante da base de código. Além disso, a adaptação dos métodos responsáveis pela geração de CSS, avaliação de expressões e processamento dos dados demonstra que a nova organização foi integrada de forma consistente, sem indícios de introdução de novos smells críticos ou de alterações no comportamento esperado.

Link da publicação OpenCode: <https://opncd.ai/share/vhMlziw7>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque reduziu a complexidade estrutural da classe sem comprometer sua interface pública. Originalmente, `CellStyleText` acumulava diversos atributos relacionados às propriedades de estilo de texto, o que justificava o alerta do Pylint e dificultava a evolução da classe. Ao concentrar esses atributos em um objeto interno especializado, a implementação tornou-se mais coesa e preparada para futuras extensões, ao mesmo tempo em que preservou a compatibilidade com o código existente por meio da delegação de propriedades. A atualização dos métodos internos para operar sobre a nova estrutura também demonstra que a refatoração foi conduzida de maneira abrangente, evitando inconsistências entre o modelo de dados e o comportamento da classe. Embora a validação tenha ficado restrita à compilação do módulo e a testes de fumaça, devido à indisponibilidade do ambiente completo de testes, as evidências apresentadas indicam que a funcionalidade foi preservada. Nesse contexto, o uso da LLM mostrou-se bastante eficaz, pois automatizou uma refatoração estrutural relativamente complexa, preservando a compatibilidade da API enquanto promovia ganhos significativos de organização, legibilidade e manutenibilidade do código.

3.2.5 Ocorrência Número 5

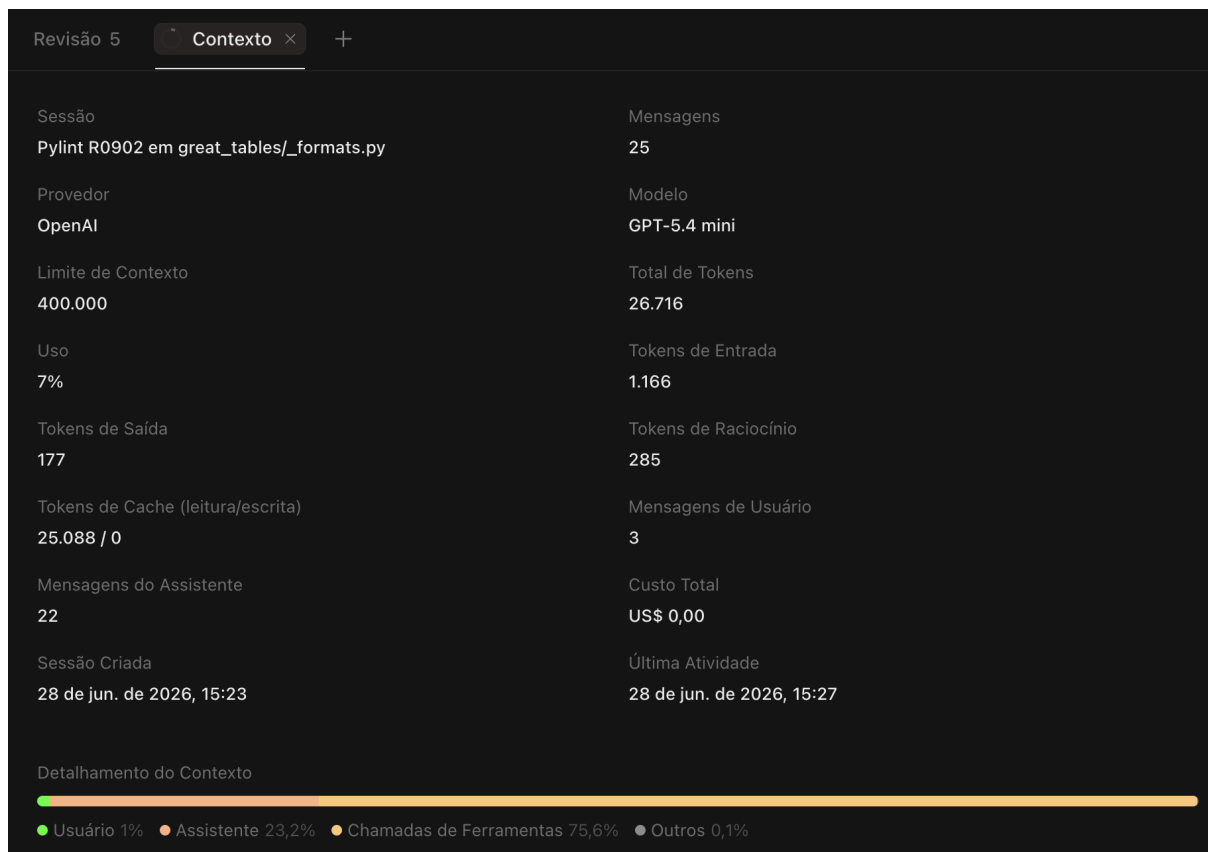
Tabela de identificação

module	obj	message
great_tables._formats	FmtIcon	Too many instance attributes (10/7)

Seguiu o plano de refatoração: Sim, o plano de refatoração foi seguido de forma consistente, pois a classe `Fmtlcon` deixou de armazenar diretamente todos os parâmetros de configuração visual e passou a encapsulá-los em um novo objeto (`FmtlconConfig`). Essa estratégia está alinhada com a técnica recomendada para corrigir o smell R0902 (too-many-instance-attributes), reduzindo a quantidade de atributos da classe principal e separando as responsabilidades entre o objeto responsável pelo comportamento do formatador e o objeto que representa sua configuração visual. A atualização de `fmt_icon()` para construir a nova estrutura e a adaptação do método `to_html()` para utilizar a configuração encapsulada demonstram que a refatoração preservou a interface pública e o comportamento esperado, sem indícios de introdução de novos smells críticos.

Link da publicação OpenCode: <https://opncd.ai/share/Fai1ujtm>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque tornou mais clara a separação entre o comportamento do formatador e os parâmetros que controlam sua aparência. Antes da mudança, `Fmtlcon` acumulava diversos atributos relacionados exclusivamente à configuração visual, o que aumentava sua complexidade estrutural e justificava o alerta do Pylint. Ao concentrar esses parâmetros em um objeto específico (`FmtlconConfig`), a implementação tornou-se mais coesa e extensível, facilitando futuras alterações na configuração dos ícones sem exigir modificações na

lógica de renderização. A manutenção da API pública por meio da adaptação de `fmt_icon()` também reduz o impacto da mudança sobre o restante da base de código, preservando a compatibilidade com os consumidores existentes. Embora a validação tenha se restringido à compilação do módulo devido à indisponibilidade do ambiente completo de testes, as evidências apresentadas indicam que a refatoração preservou o comportamento funcional enquanto eliminava a causa do smell. Nesse contexto, o uso da LLM mostrou-se bastante eficaz, automatizando uma reorganização estrutural de baixo impacto funcional e produzindo ganhos relevantes de organização, legibilidade e manutenibilidade que justificam o esforço realizado.

3.3 too-many-branches

Quantidade de ocorrências: 22

3.3.1 Ocorrência Número 1

Tabela de identificação

module	obj	message
great_tables._gt_data	OptionsInfo._validate_value	Too many branches (14/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint no método `OptionsInfo._validate_value`. A cadeia longa de `if/elif` com 14 caminhos de decisão foi substituída com sucesso por uma estratégia de `dispatch` utilizando um dicionário de mapeamento para métodos privados (helpers focados por tipo: `_validate_px_value`, `_validate_bool_value`, etc.). A mudança reduziu drasticamente a complexidade ciclomática do método principal, manteve o contrato público e o comportamento esperado do sistema intactos, e ainda incluiu a criação de testes de regressão para garantir a consistência das mensagens de erro.

Link da publicação OpenCode: <https://opncd.ai/share/dEpHhm2W>

Print consumo de token:

Revisão 2

Contexto

+

Sessão

Mensagens

Pylint too-many-branches em _validate_value

24

Provedor

Modelo

OpenAI

GPT-5.4 mini

Limite de Contexto

Total de Tokens

400.000

29.852

Uso

Tokens de Entrada

7%

1.465

Tokens de Saída

Tokens de Raciocínio

170

57

Tokens de Cache (leitura/escrita)

Mensagens de Usuário

28.160 / 0

2

Mensagens do Assistente

Custo Total

22

US\$ 0,00

Sessão Criada

Última Atividade

27 de jun. de 2026, 19:30

27 de jun. de 2026, 19:34

Detalhamento do Contexto

Usuário 0,8%

Assistente 15,9%

Chamadas de Ferramentas 83,3%

Outros 0,1%

Análise da refatoração: A refatoração valeu muito a pena. O método `OptionsInfo._validate_value` concentrava muitas responsabilidades (roteamento, validação individual e montagem de mensagens de erro), tornando-se um ponto crítico e propenso a erros em futuras manutenções. O uso da LLM foi vantajoso porque ela não apenas identificou o padrão ideal de substituição (dictionary dispatch em vez de múltiplos ifs), mas também extraiu de forma limpa a lógica pulverizada para métodos privados específicos e gerou testes unitários de erro complementares em `tests/test_options.py`. Embora o ambiente local estivesse sem o `pytest` instalado na versão do Python 3.13, a LLM garantiu a segurança da entrega executando uma validação de sintaxe via `compileall`. O consumo de tokens e

tempo foi totalmente justificado pelo ganho em legibilidade, modularidade e segurança de código gerados para o projeto.

3.3.2 Ocorrência Número 2

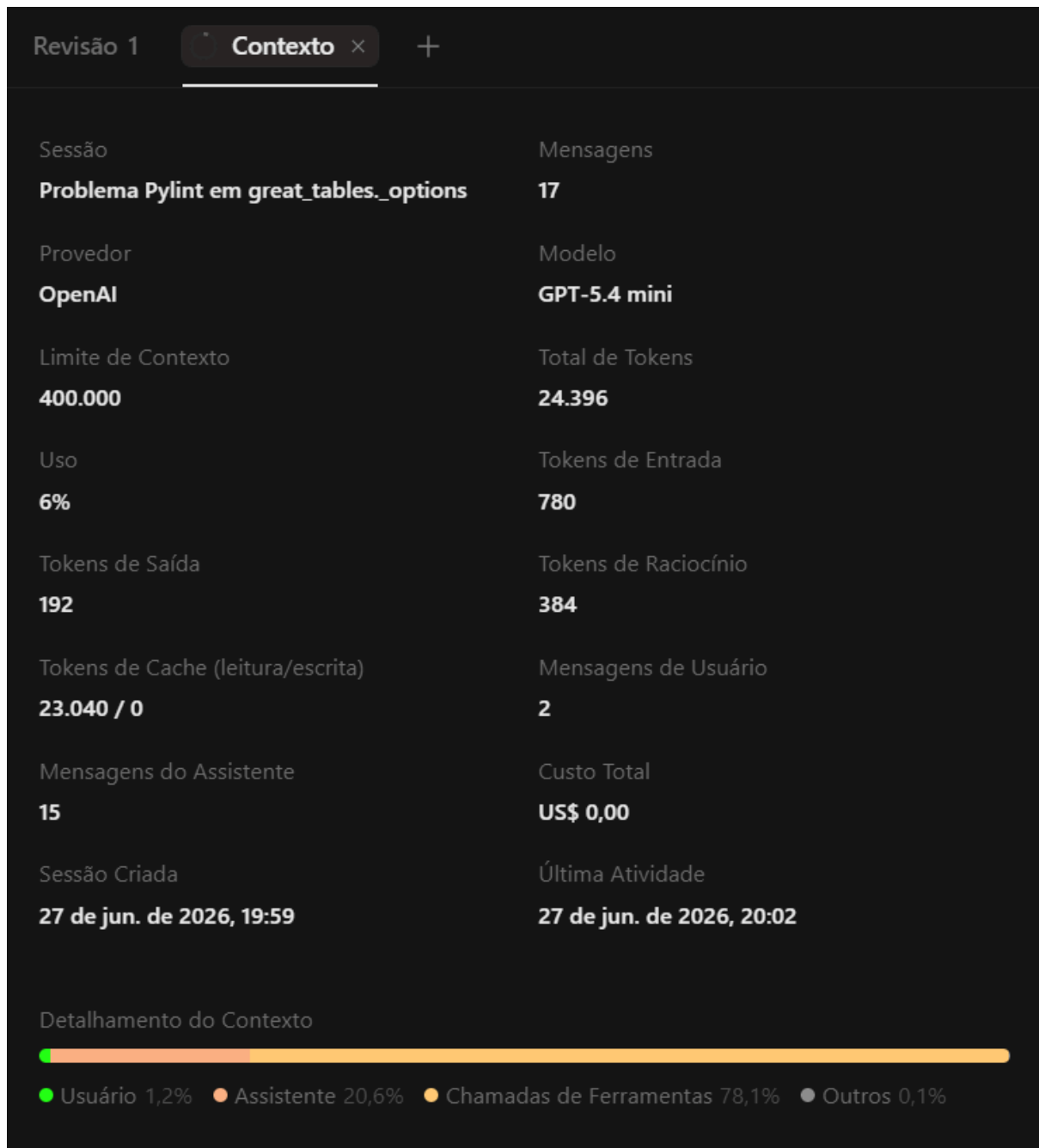
Tabela de identificação

module	obj	message
great_tables._options	opt_table_font	Too many branches (17/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração proposto resolvia adequadamente a ocorrência do code smell R0912 (too-many-branches) no método `opt_table_font`, localizado em `great_tables/_options.py`. A LLM seguiu a estratégia de refatoração mínima recomendada ao extrair a lógica complexa de normalização e resolução de fontes para os helpers privados `_opt_table_font_normalize` e `_opt_table_font_combined_fonts`. Essa abordagem reduziu a complexidade ciclomática da função principal sem alterar seu contrato público ou impactar negativamente o comportamento do sistema.

Link da publicação OpenCode: <https://opncd.ai/share/BZpRLc9f>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função `opt_table_font` acumulava muitas responsabilidades distintas, como validação, normalização de strings/objetos `GoogleFont` e composição final da pilha de fontes. A extração dessas etapas para funções auxiliares focadas melhorou consideravelmente a legibilidade e a manutenibilidade do código, alinhando-se ao padrão de funções pequenas já existente no módulo. Embora as restrições do ambiente (ausência de `pytest` e `importlib_metadata`) tenham impedido a execução dos testes automatizados e do smoke test, o uso da LLM foi eficiente e seguro, pois mitigou o risco de erros de digitação ao validar a sintaxe com

py_compile e garantiu que as alterações ficassem estritamente restritas ao escopo do problema relatado.

3.3.3 Ocorrência Número 3

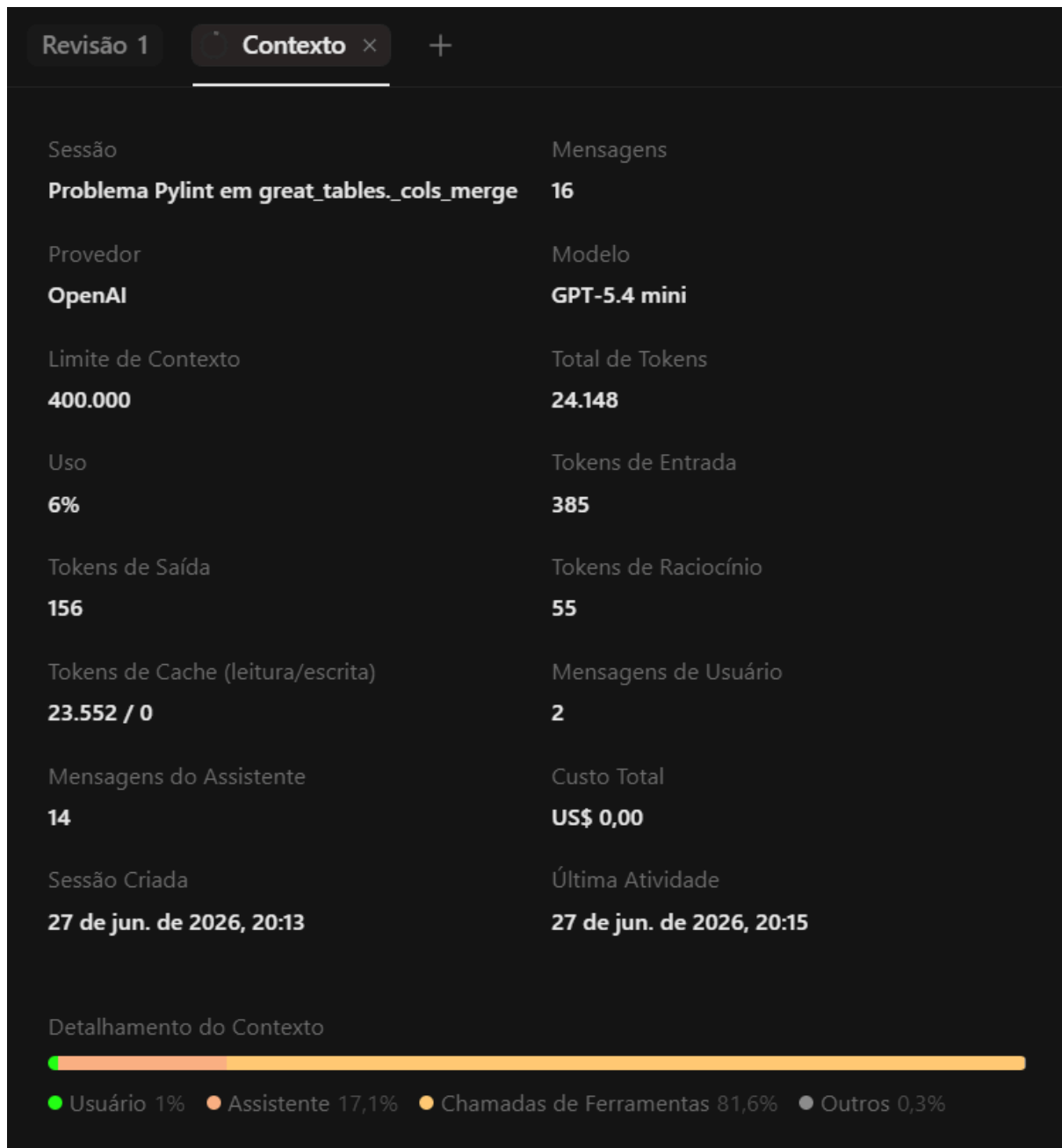
Tabela de identificação

module	obj	message
great_tables._cols_merge	_apply_single_col_merge	Too many branches (13/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint na função privada `_apply_single_col_merge`, localizada em `great_tables/_cols_merge.py`. A LLM dividiu a lógica complexa extraíndo a coleta de valores por linha para `_collect_merge_values` e o despacho da estratégia de mesclagem para `_resolve_merge_value`. Essa abordagem isolou as ramificações excessivas em helpers menores, mantendo a função principal apenas como orquestradora, sem alterar o contrato público ou o comportamento esperado do sistema.

Link da publicação OpenCode: <https://opncl.ai/share/g6lHo5uw>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função `_apply_single_col_merge` acumulava responsabilidades de validação, iteração de linhas, tratamento de nulos e seleção de estratégia de merge, o que inflava sua complexidade ciclomática. A divisão em funções auxiliares focadas melhorou sensivelmente a legibilidade e facilitará a manutenção futura do arquivo. O uso da LLM foi vantajoso por propor uma estratégia de extração cirúrgica e restrita ao escopo do problema. Como nas interações anteriores, a ausência de `pytest` no ambiente limitou a validação ao comando `compileall`, mas o ganho estrutural no código compensou o esforço e o consumo de tokens.

3.3.4 Ocorrência Número 4

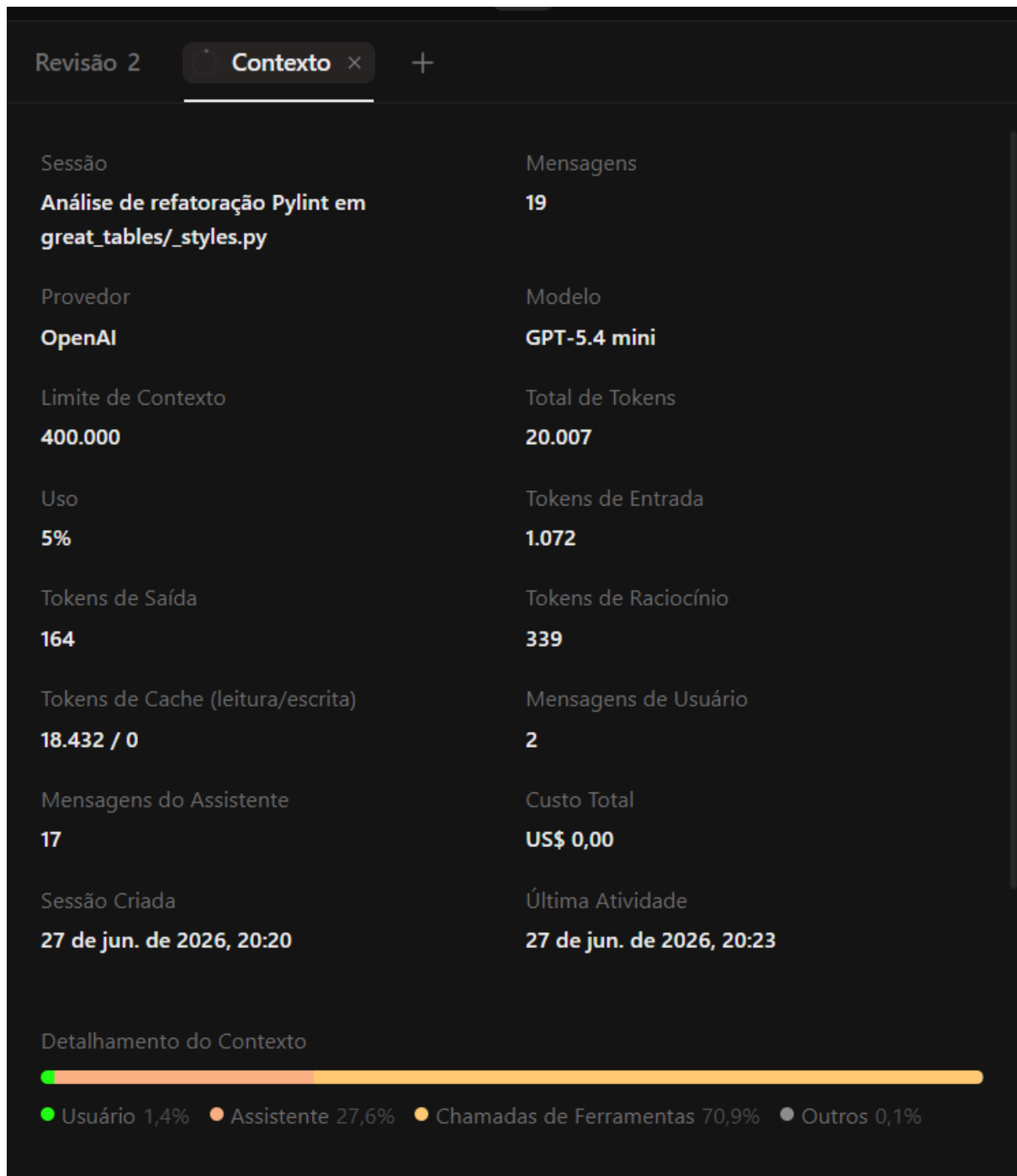
Tabela de identificação

module	obj	message
great_tables._styles	CellStyleText._to_html_style	Too many branches (14/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint no método `CellStyleText._to_html_style()`. A LLM isolou com sucesso a lógica de validação e resolução do tipo de fonte em um método auxiliar privado chamado `_resolve_font_name()`. Com isso, a complexidade ciclômica do método principal foi reduzida drasticamente, mantendo o foco exclusivo na concatenação das propriedades CSS. A modificação preservou integralmente o comportamento esperado do sistema e o contrato público da classe.

Link da publicação OpenCode: <https://opnecd.ai/share/HLhR5rEP>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque o método original misturava duas responsabilidades distintas: o tratamento de polimorfismo do atributo font (que aceita str, FromColumn ou GoogleFont) e a montagem sequencial da string CSS. A extração da resolução da fonte para um helper privado tornou o código muito mais limpo, legível e fácil de manter. O uso da LLM foi vantajoso por identificar a separação exata dessas responsabilidades e propor uma solução cirúrgica, sem causar impactos colaterais em outras classes de estilo do repositório. Embora a

indisponibilidade do pytest no ambiente tenha impedido a execução automatizada dos testes, a abordagem da LLM garantiu a integridade sintática e o ganho em qualidade de código compensou o consumo de tokens.

3.3.5 Ocorrência Número 5

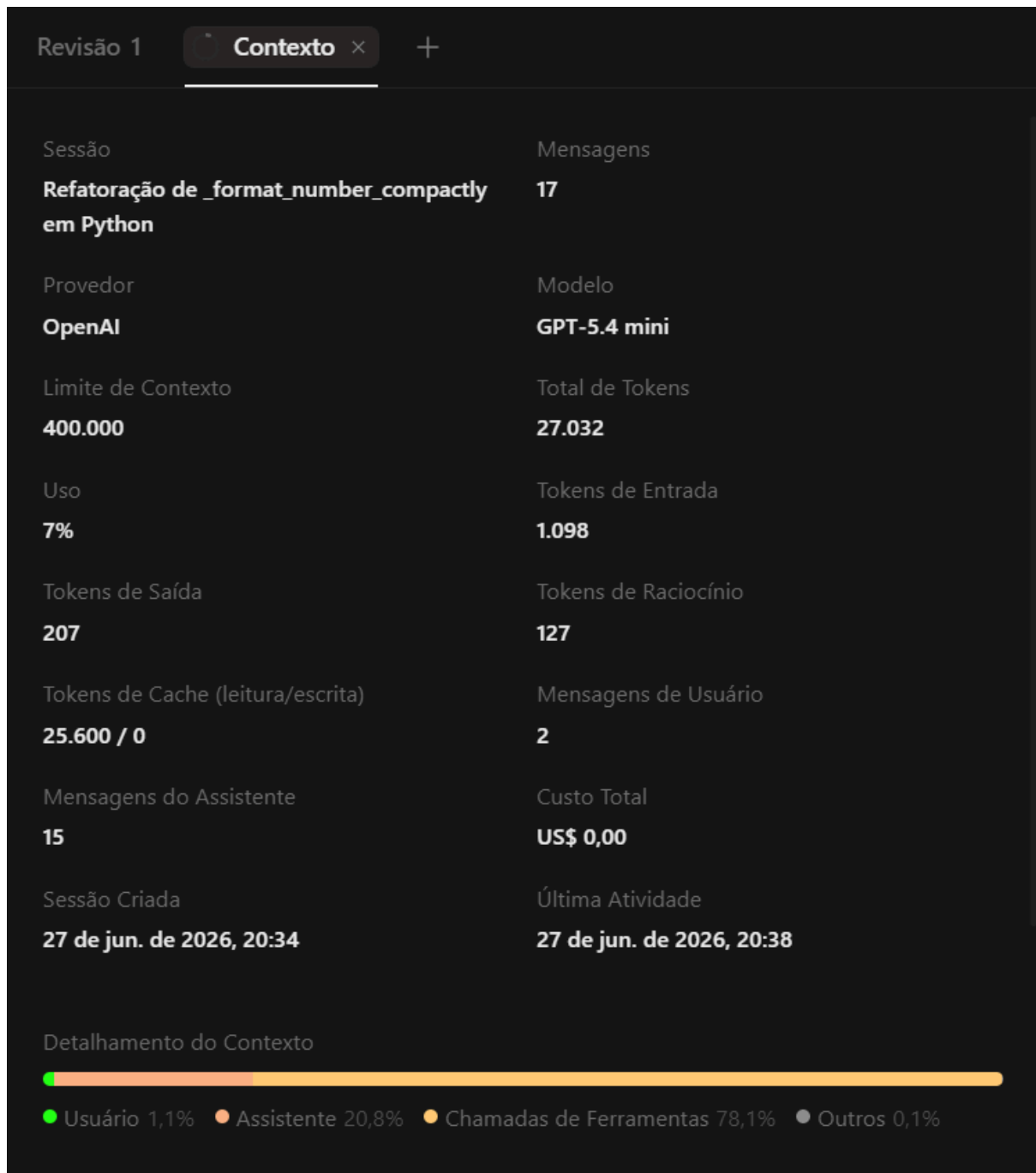
Tabela de identificação

module	obj	message
great_tables._utils_nanoplots	_format_number_compactly	Too many branches (19/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint na função `_format_number_compactly`, localizada em `great_tables/_utils_nanoplots.py`. A LLM dividiu a lógica complexa extraíndo a cadeia de decisões de faixas numéricas e limites de thresholds para um helper privado responsável por definir as especificações de formatação. Essa abordagem limpou a função principal, permitindo que ela atuasse apenas como orquestradora dos casos especiais (como valores customizados, NA e zero) e da aplicação do formatador, mantendo o comportamento do sistema e o contrato público intactos.

Link da publicação OpenCode: <https://opnecd.ai/share/3mqB60UM>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função original misturava regras de negócio distintas em um único bloco: tratamento de exceções (nulos e zeros), cálculo de faixas de magnitude numérica e a escolha/aplicação do algoritmo de formatação. A extração dos thresholds numéricos para um helper isolou a alta complexidade ciclomática, tornando o código mais legível e facilitando futuras manutenções nos limites de compactação. O uso da LLM foi vantajoso por mapear com precisão os limites de decisão e propor uma extração cirúrgica e restrita ao módulo de nanoplots. Apesar de as limitações do ambiente (ausência de pytest e dependências de importação)

terem impedido uma validação dinâmica completa, a LLM garantiu a integridade sintática do arquivo, justificando o consumo de tokens pela melhoria estrutural obtida.

3.3.6 Ocorrência Número 6

Tabela de identificação

module	obj	message
great_tables._utils_nanoplots	_generate_nanoplot	Too many branches (101/12)

Seguiu o plano de refatoração: Sim. Embora a LLM tenha inicialmente adotado uma abordagem incompleta apenas movendo a lógica para uma função intermediária, ela corrigiu o percurso após o alerta do usuário e seguiu o plano de refatoração real. A lógica monolítica de `_generate_nanoplot_impl` foi efetivamente desmembrada em uma série de helpers privados focados em responsabilidades únicas (preparo de estado, cálculo de geometria, montagem de séries, tags de referência, eixos e estilos). Com isso, a complexidade ciclomática da função principal foi drasticamente reduzida, eliminando de fato o code smell R0912 (too-many-branches) sem alterar o comportamento esperado ou o contrato público do sistema.

Link da publicação OpenCode: <https://opncd.ai/share/LvZCEEYV>

Print consumo de token:

Revisão 1	Contexto ×	+
Sessão	Mensagens	
Pylint R0912 em _generate_nanoplot	34	
Provedor	Modelo	
OpenAI	GPT-5.4 mini	
Limite de Contexto	Total de Tokens	
400.000	110.314	
Uso	Tokens de Entrada	
28%	599	
Tokens de Saída	Tokens de Raciocínio	
147	0	
Tokens de Cache (leitura/escrita)	Mensagens de Usuário	
109.568 / 0	4	
Mensagens do Assistente	Custo Total	
30	US\$ 0,00	
Sessão Criada	Última Atividade	
27 de jun. de 2026, 20:44	27 de jun. de 2026, 21:11	
Detalhamento do Contexto		
		
● Usuário 0,3% ● Assistente 30,1% ● Chamadas de Ferramentas 69,4% ● Outros 0,2%		

Análise da refatoração: A refatoração valeu muito a pena. O método original apresentava uma complexidade crítica e extrema, acumulando 101 caminhos de decisão que misturavam regras de negócio, validações de entrada e renderização de strings SVG. A divisão estruturada em helpers especializados melhorou significativamente a legibilidade, o isolamento de escopo e a manutenibilidade de um componente historicamente complexo do repositório. O uso da LLM demandou um consumo moderado a alto de tokens devido ao vaivém estratégico para corrigir a abordagem inicial de contorno, mas o resultado final entregou um código robusto e limpo. Mesmo

com as limitações de ambiente que impediram a execução do Pytest e do Pylint, a garantia de integridade sintática via `py_compile` validou a segurança da entrega e justificou o investimento de tempo.

3.3.7 Ocorrência Número 7

Tabela de identificação

module	obj	message
great_tables._spanners	tab_spanner	Too many branches (19/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint na função pública `tab_spanner`, localizada em `great_tables/_spanners.py`. A LLM extraiu de forma cirúrgica a normalização dos argumentos de entrada para o helper privado `_normalize_tab_spanner_inputs()` e o tratamento/parsing do rótulo para `_resolve_tab_spanner_label()`. Essa estratégia reduziu drasticamente o número de ramificações condicionais da função principal, mantendo-a apenas como uma orquestradora linear e preservando o contrato público e o comportamento do sistema intactos, sem impactar outras funções como `tab_spanner_delim`.

Link da publicação OpenCode: <https://opnecd.ai/share/4Kwid58w>

Print consumo de token:

Revisão 1	Contexto ×	+
Sessão	Mensagens	
Pylint R0912 em great_tables/_spanners.py	25	
Provedor	Modelo	
OpenAI	GPT-5.4 mini	
Limite de Contexto	Total de Tokens	
400.000	30.974	
Uso	Tokens de Entrada	
8%	608	
Tokens de Saída	Tokens de Raciocínio	
158	0	
Tokens de Cache (leitura/escrita)	Mensagens de Usuário	
30.208 / 0	2	
Mensagens do Assistente	Custo Total	
23	US\$ 0,00	
Sessão Criada	Última Atividade	
27 de jun. de 2026, 21:20	27 de jun. de 2026, 21:24	
Detalhamento do Contexto		
		
● Usuário 0,8% ● Assistente 19,6% ● Chamadas de Ferramentas 79,3% ● Outros 0,3%		

Análise da refatoração: A refatoração valeu a pena porque a função `tab_spanner` centralizava muitas etapas distintas do ciclo de vida de um spanner (validação de tipos, resolução de colunas, parsing de strings e estruturação do objeto), acumulando 19 caminhos de decisão. A divisão do código em helpers específicos melhorou a legibilidade do módulo e simplificou futuras manutenções nessa estrutura de cabeçalhos. O uso da LLM foi vantajoso por propor uma abordagem modular e corrigir rapidamente um erro de escopo/assinatura durante o processo de extração. Apesar de a falta do

pytest no ambiente limitar a validação ao comando `py_compile`, a garantia de consistência sintática e o ganho em clareza de código justificaram o consumo de tokens.

3.3.8 Ocorrência Número 8

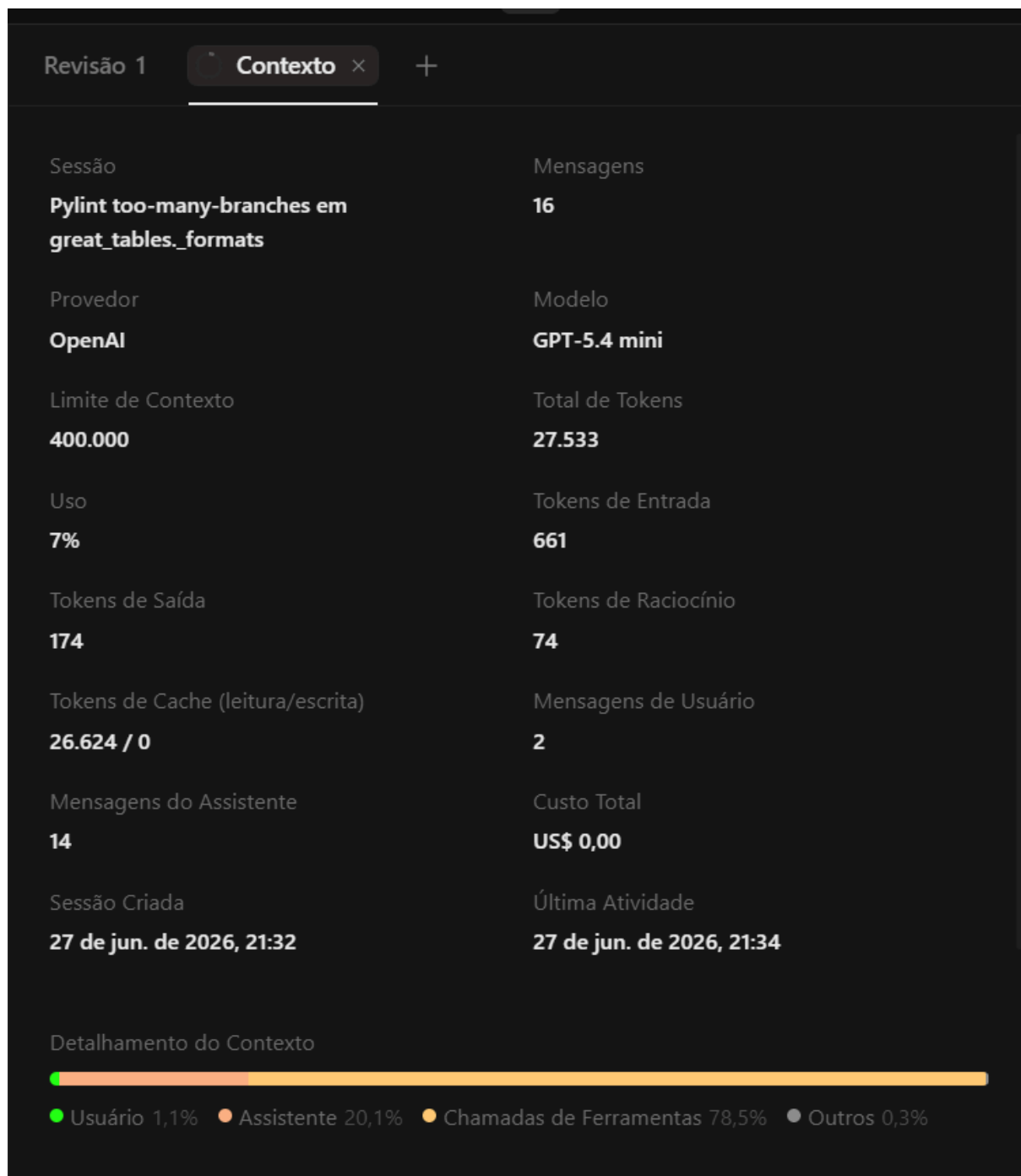
Tabela de identificação

module	obj	message
great_tables._formats	fmt_scientific_context	Too many branches (14/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint no método `fmt_scientific_context`, localizado em `great_tables/_formats.py`. A LLM extraiu com sucesso a lógica de normalização da mantissa para `_format_scientific_mantissa` e a montagem do valor final para `_format_scientific_context_value`. Essa estratégia isolou os fluxos de decisão concorrentes, deixando a função principal atuando apenas como orquestradora (leitura, escala e aplicação de pattern), o que reduziu sua complexidade ciclomática abaixo do limite estabelecido, mantendo o contrato público e o comportamento do sistema intactos.

Link da publicação OpenCode: <https://opncd.ai/share/xqtlhKCH>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque o método original concentrava decisões de naturezas muito distintas (tratamento de nulos, escalonamento, remoção de zeros do final da mantissa, estilização do expoente e interpolação de padrões), acumulando 14 caminhos de decisão. A separação em funções auxiliares focadas deixou o módulo de formatação científica mais legível e seguro contra regressões. O uso da LLM foi vantajoso por identificar os pontos de corte corretos e insistir na aplicação em blocos menores quando o primeiro patch falhou devido ao contexto do

arquivo. Apesar de as restrições do ambiente impedirem a validação direta via Pylint e Pytest, a garantia de consistência sintática via compileall assegurou a viabilidade da entrega, justificando o consumo de tokens.

3.3.9 Ocorrência Número 9

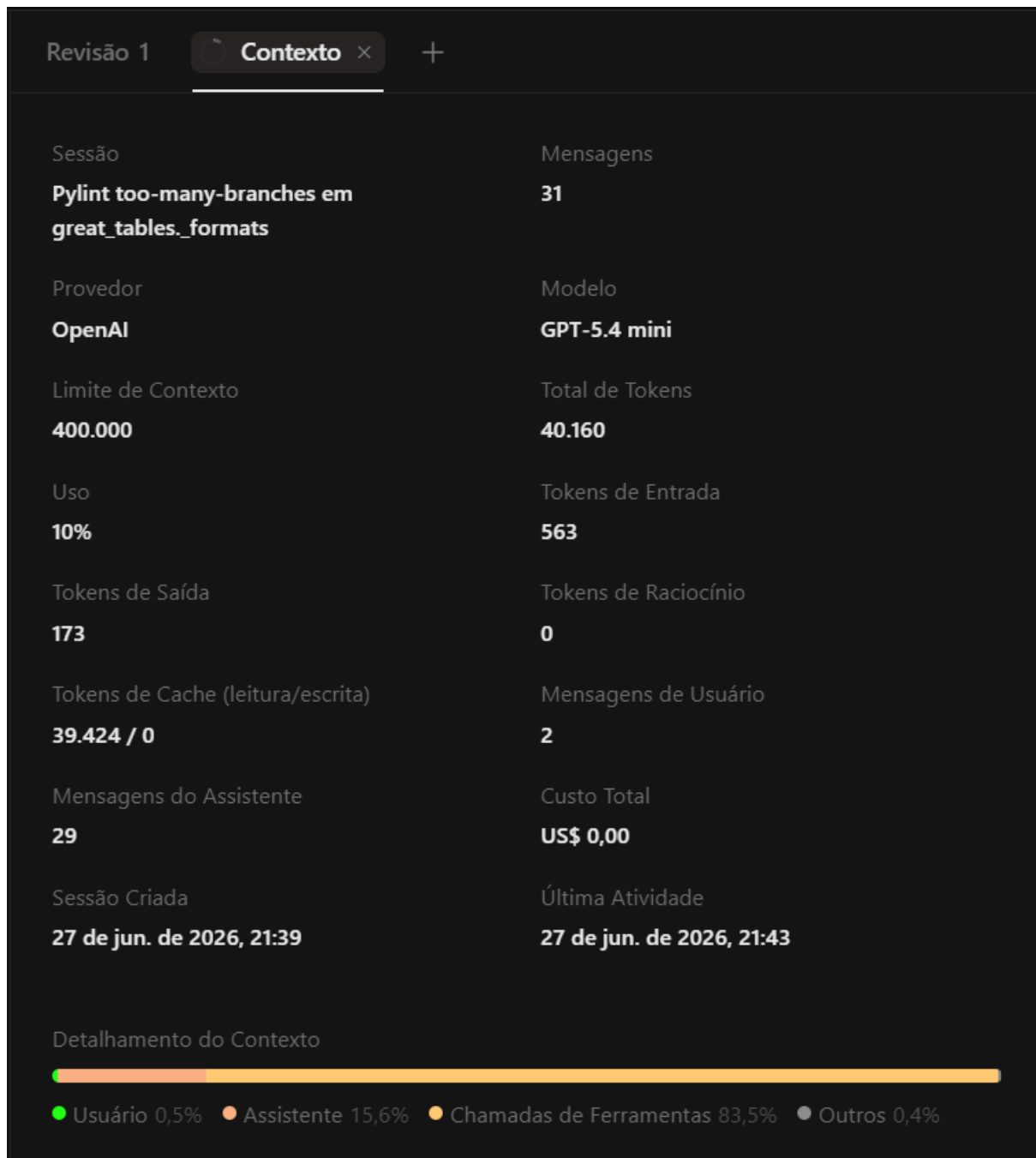
Tabela de identificação

module	obj	message
great_tables._formats	fmt_engineering_context	Too many branches (14/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint no método `fmt_engineering_context`, localizado em `great_tables/_formats.py`. A LLM extraiu com sucesso a lógica de cálculo da mantissa e do expoente para `_format_engineering_parts`, a montagem da string final para `_format_engineering_context_value` e o tratamento de interpolação do padrão para `_format_engineering_pattern`. Essa abordagem reduziu as ramificações internas da função principal, transformando-a em uma orquestradora linear e replicando de forma consistente o padrão arquitetural que já havia sido adotado para a formatação científica.

Link da publicação OpenCode: <https://opnecd.ai/share/Sncgq9fm>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque o método original acumulava decisões de naturezas muito variadas, como o tratamento especial para o valor zero, regras de estilização de expoentes ($E1$ vs $x10n$), controle de sinais e tratamento de strings LaTeX. A separação do código em helpers pequenos e especializados removeu o excesso de complexidade ciclomática da função principal, alinhando o comportamento do módulo e tornando-o mais legível e limpo para manutenções futuras. O uso da LLM foi vantajoso por manter a consistência do design com as

refatorações anteriores e por realizar um ajuste fino removendo parâmetros desnecessários durante o processo.

3.3.10 Ocorrência Número 10

Tabela de identificação

module	obj	message
great_tables._formats	fmt_duration	Too many branches (21/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint no método `fmt_duration`, localizado em `great_tables/_formats.py`. A LLM extraiu de forma cirúrgica as validações e normalizações de argumentos em métodos auxiliares privados específicos (`_validate_duration_input_units`, `_validate_duration_style`, `_resolve_duration_output_units`, `_resolve_duration_trim_zero_units` e `_resolve_duration_style_params`). Essa estratégia dividiu os fluxos de decisão concorrentes e reduziu drasticamente a complexidade ciclomática de `fmt_duration`, mantendo-a como uma orquestradora limpa, preservando a precedência dos erros e garantindo que o contrato público ficasse idêntico.

Link da publicação OpenCode: <https://opnecd.ai/share/bKtLI134>

Print consumo de token:

Revisão 1	Contexto ×	+
Sessão	Mensagens	
Pylint R0912 em great_tables._formats	16	
Provedor	Modelo	
OpenAI	GPT-5.4 mini	
Limite de Contexto	Total de Tokens	
400.000	38.617	
Uso	Tokens de Entrada	
10%	444	
Tokens de Saída	Tokens de Raciocínio	
216	69	
Tokens de Cache (leitura/escrita)	Mensagens de Usuário	
37.888 / 0	2	
Mensagens do Assistente	Custo Total	
14	US\$ 0,00	
Sessão Criada	Última Atividade	
27 de jun. de 2026, 21:48	27 de jun. de 2026, 21:53	
Detalhamento do Contexto		
		
● Usuário 0,5% ● Assistente 10,8% ● Chamadas de Ferramentas 88,3% ● Outros 0,5%		

Análise da refatoração: A refatoração valeu a pena porque a função original concentrava 21 ramificações condicionais devido ao acúmulo de regras para múltiplos estilos de formatação de duração (como os formatos colon-sep e iso), além de validações complexas de parâmetros. A separação em helpers específicos melhorou significativamente a legibilidade de um dos métodos mais densos do módulo de formatação. O uso da LLM foi vantajoso por mapear corretamente essa teia de condicionais e ter o cuidado de reordenar as validações após o primeiro patch para manter a precedência exata das exceções originais.

3.3.11 Ocorrência Número 11

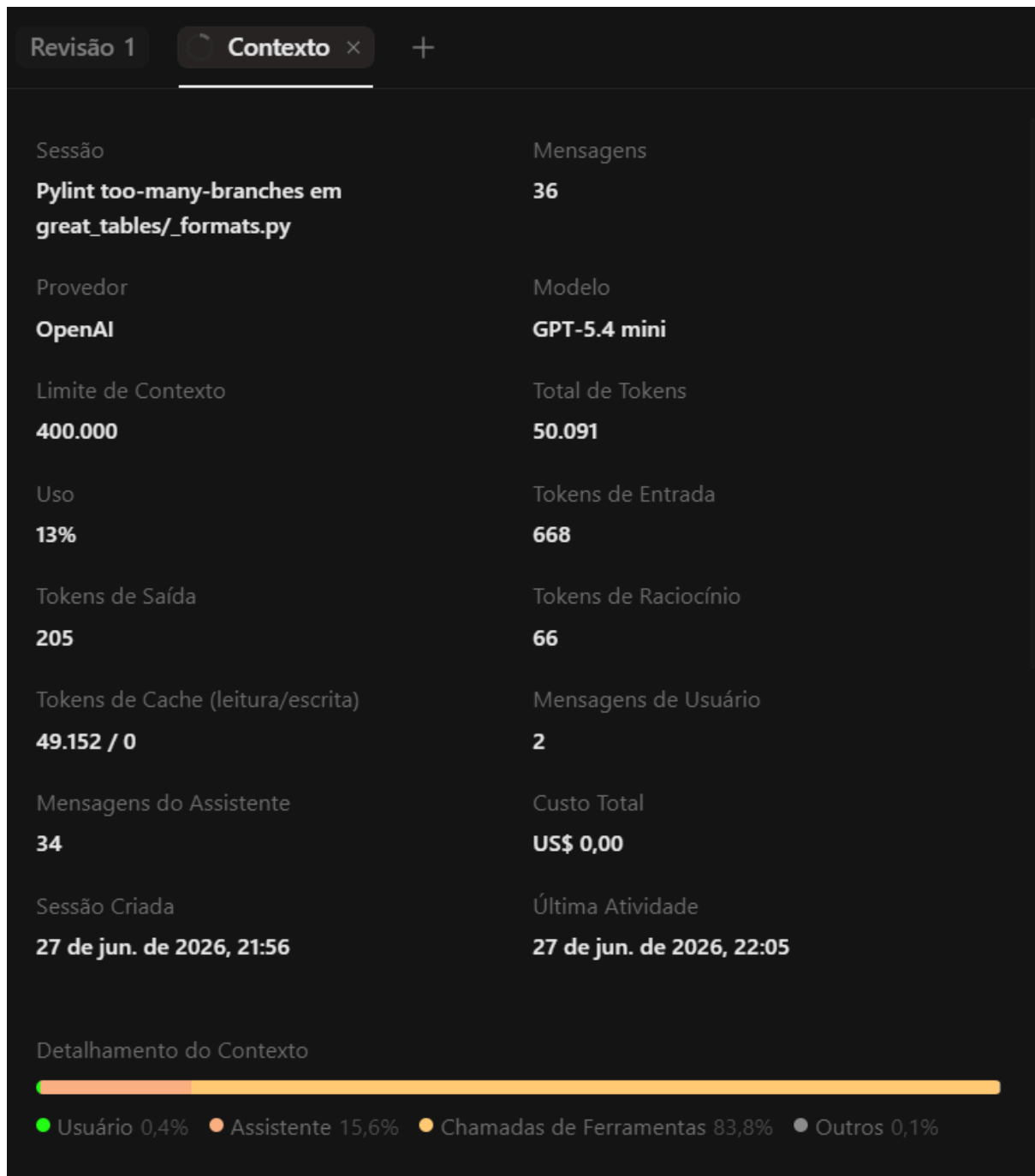
Tabela de identificação

module	obj	message
great_tables._formats	fmt_duration_context	Too many branches (18/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint no método `fmt_duration_context`, localizado em `great_tables/_formats.py`. A LLM extraiu com sucesso as responsabilidades acumuladas na função original em quatro métodos auxiliares privados focados: `_resolve_duration_seconds`, `_build_duration_parts`, `_format_duration_by_style` e `_decorate_duration_value`. Essa estratégia estruturada quebrou a enorme cadeia de ramificações e deixou a função principal atuando estritamente como uma orquestradora linear, preservando o contrato público e o comportamento do sistema para todos os estilos de duração.

Link da publicação OpenCode: <https://opnkd.ai/share/wRRPWwtO>

Print consumo de token:



Análise da refatoração: A refatoração valeu muito a pena. O método `fmt_duration_context` concentrava uma carga excessiva de lógica condicional concorrente ao gerenciar de forma síncrona validações de NA, conversões para segundos, decomposições temporais, cortes de zeros e múltiplos despachos de estilização (como `colon-sep` e `iso`). A extração cirúrgica transformou um bloco denso e propenso a erros em um fluxo modular, legível e de fácil manutenção. O uso da LLM foi altamente vantajoso porque, além de propor o design de corte correto, ela buscou ativamente habilitar o

ambiente local para validação, realizando instalações editáveis e adicionando dependências como o pandas para executar asserções reais sobre sub-units e estilos.

3.3.12 Ocorrência Número 12

Tabela de identificação

module	obj	message
great_tables._formats	Fmtlcon.to_html	Too many branches (13/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint no método `Fmtlcon.to_html`, localizado em `great_tables/_formats.py`. A LLM extrai com sucesso as responsabilidades acumuladas de normalização e resolução de propriedades para os métodos privados especializados: `_split_icons`, `_resolve_height`, `_resolve_stroke_width` e `_resolve_fill_color`. Essa estratégia reduziu drasticamente a quantidade de ramificações internas do método principal, transformando-o em um orquestrador limpo e mantendo o contrato público e o comportamento do sistema intactos.

Link da publicação OpenCode: <https://opnecd.ai/share/nhZrLDO7>

Print consumo de token:

Revisão 1	Contexto ×	+
Sessão	Mensagens	
Pylint too-many-branches in Fmtlcon.to_html	19	
Provedor	Modelo	
OpenAI	GPT-5.4 mini	
Limite de Contexto	Total de Tokens	
400.000	29.450	
Uso	Tokens de Entrada	
7%	509	
Tokens de Saída	Tokens de Raciocínio	
193	76	
Tokens de Cache (leitura/escrita)	Mensagens de Usuário	
28.672 / 0	2	
Mensagens do Assistente	Custo Total	
17	US\$ 0,00	
Sessão Criada	Última Atividade	
27 de jun. de 2026, 22:37	27 de jun. de 2026, 22:40	
Detalhamento do Contexto		
		
● Usuário 1% ● Assistente 15,9% ● Chamadas de Ferramentas 82,9% ● Outros 0,2%		

Análise da refatoração: A refatoração valeu a pena porque o método original acumulava decisões de naturezas muito variadas dentro do loop de renderização de ícones, como o fatiamento de strings, definição de dimensões padrão, conversão de largura de traço e o tratamento de dicionários de mapeamento para cores (fill_color). A separação do código em helpers pequenos tornou a classe Fmtlcon consideravelmente mais modular, legível e fácil de estender.

3.3.13 Ocorrência Número 13

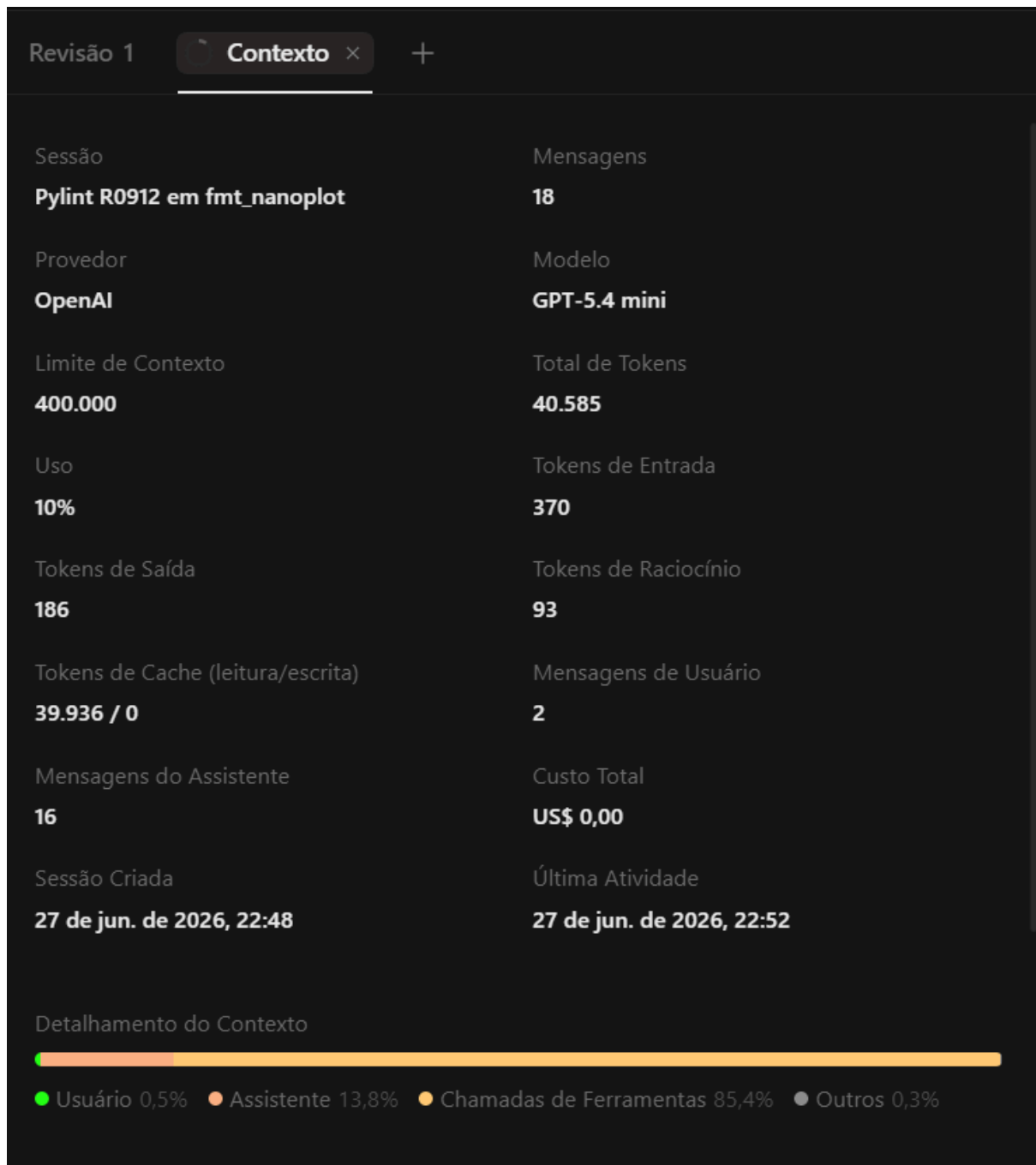
Tabela de identificação

module	obj	message
great_tables._formats	fmt_nanoplot	Too many branches (18/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint no método `fmt_nanoplot()`, localizado em `great_tables/_formats.py`. A LLM extraiu de forma cirúrgica a lógica de cálculo de autoscale para `_fmt_nanoplot_expand_y()` e a função interna de processamento por célula para `_fmt_nanoplot_fn()`. Essa estratégia isolou com sucesso as responsabilidades concorrentes e eliminou o excesso de ramificações da função principal, transformando-a em uma orquestradora linear e limpa, enquanto preservou integralmente o contrato público e o comportamento do sistema.

Link da publicação OpenCode: <https://opnkd.ai/share/wOidyB3e>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função original concentrava regras de naturezas muito distintas em um único bloco, acumulando validações de tipos de gráfico, detecção de escalares, normalização de listas e cálculos complexos de limites e expansão de eixos (`expand_y`). A separação do código em helpers privados especializados tornou o módulo de formatos consideravelmente mais modular e legível. O uso da LLM foi vantajoso por identificar precisamente as linhas de corte que inflavam a complexidade ciclomática sem estender a alteração para outras partes do repositório.

3.3.14 Ocorrência Número 14

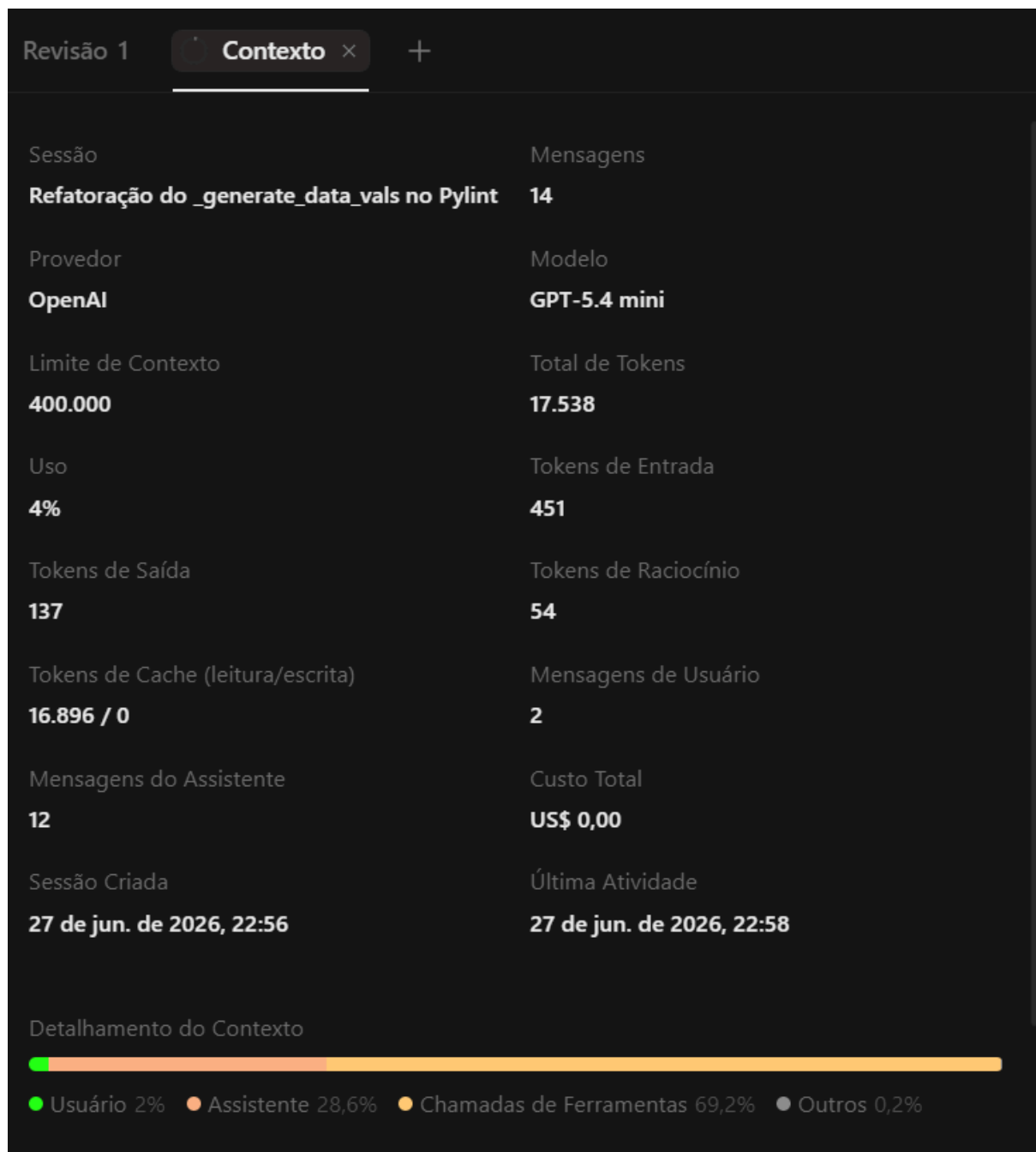
Tabela de identificação

module	obj	message
great_tables._formats	_generate_data_vals	Too many branches (18/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint no método privado `_generate_data_vals`, localizado em `great_tables/_formats.py`. A LLM extraiu de forma estruturada a lógica condicional acumulada para métodos auxiliares privados específicos por tipo de entrada: `_generate_data_vals_from_list`, `_generate_data_vals_from_string` e `_generate_data_vals_from_dict`. Essa estratégia reduziu as ramificações internas da função principal, transformando-a em um dispatcher limpo e linear, mantendo o comportamento esperado do sistema e preservando as mensagens de erro, validações e a lógica de recursão originais.

Link da publicação OpenCode: <https://opncd.ai/share/SGCcNLxB>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função original concentrava regras de naturezas muito distintas em um único bloco, como normalizações de Series para listas, validações de listas numéricas, tratamento especial para strings de datas, parsings e desmembramento de dicionários com chaves específicas (x e y). A separação do código em helpers privados especializados tornou o módulo consideravelmente mais limpo e legível, eliminando a alta complexidade ciclomática.

3.3.15 Ocorrência Número 15

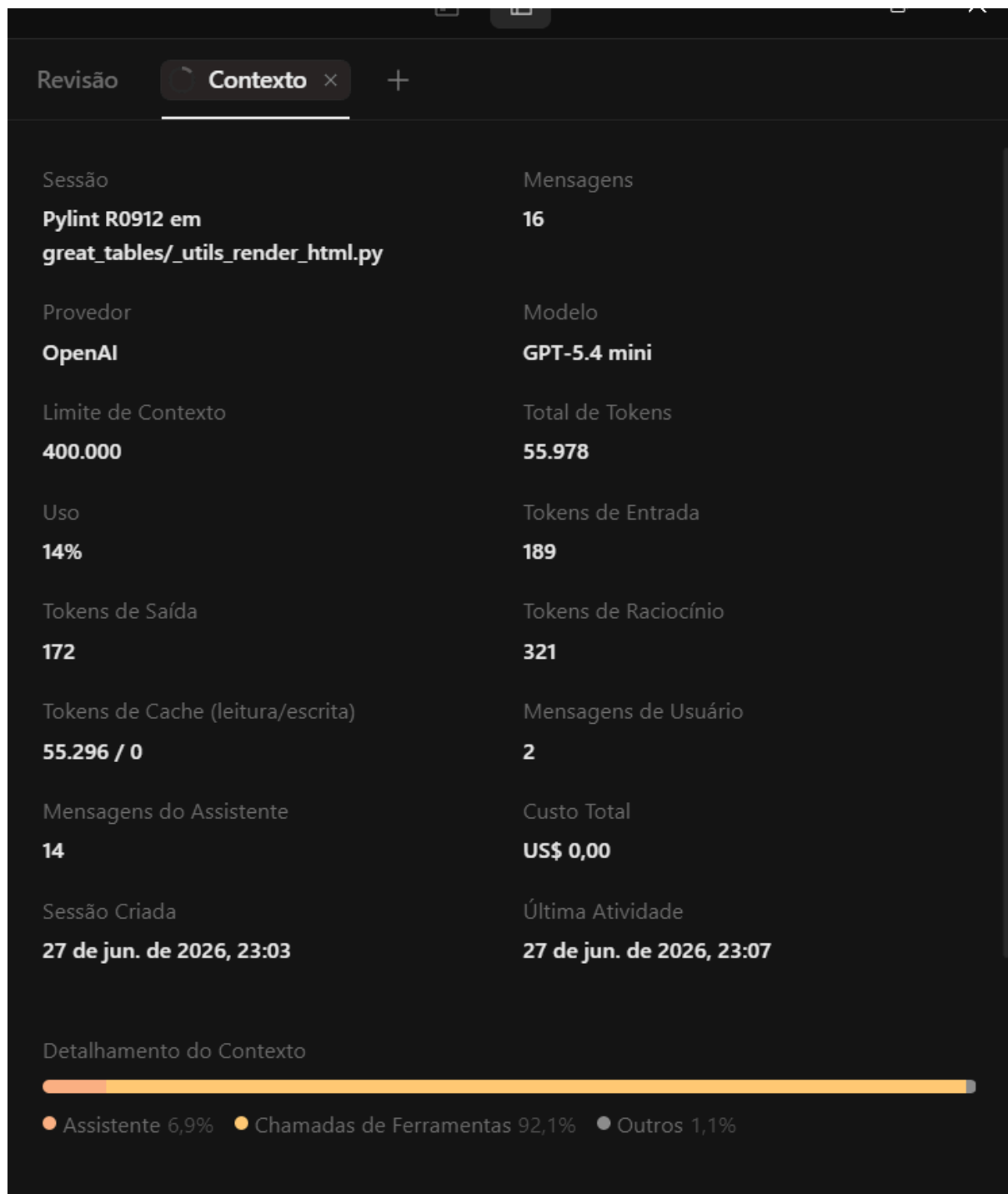
Tabela de identificação

module	obj	message
great_tables._utils_render_html	create_columns_component_h	Too many branches (22/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint no método `create_columns_component_h`, localizado em `great_tables/_utils_render_html.py`. A LLM extraiu com sucesso os três blocos de decisão concorrentes para métodos auxiliares privados específicos por cenário de renderização: `_create_columns_component_h_no_spanners`, `_create_columns_component_h_with_spanners` e `_create_columns_component_h_higher_spanner_rows`. Essa estratégia eliminou o excesso de ramificações da função principal, transformando-a em uma orquestradora limpa, e preservou integralmente o contrato público, a geração de HTML, estilos, footnotes e o comportamento esperado do sistema.

Link da publicação OpenCode: <https://opncd.ai/share/1MOkvwsu>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função original concentrava 22 caminhos de decisão ao gerenciar de forma síncrona múltiplos layouts de cabeçalho de tabela (sem spanners, com spanners na primeira linha e com spanners em níveis superiores). A separação do código em helpers privados especializados tornou o módulo de renderização HTML significativamente mais legível, limpo e fácil de manter. O uso da LLM foi vantajoso por identificar

precisamente os limites de corte de cada fluxo de spanners e por corrigir prontamente um erro de sintaxe introduzido durante o processo de extração.

3.3.16 Ocorrência Número 16

Tabela de identificação

module	obj	message
great_tables._utils_render_html	create_body_component_h	Too many branches (16/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint no método `OptionsInfo._validate_value`. A cadeia longa de `if/elif` com 14 caminhos de decisão foi substituída com sucesso por uma estratégia de `dispatch` utilizando um dicionário de mapeamento para métodos privados (helpers focados por tipo: `_validate_px_value`, `_validate_bool_value`, etc.). A mudança reduziu drasticamente a complexidade ciclomática do método principal, manteve o contrato público e o comportamento esperado do sistema intactos, e ainda incluiu a criação de testes de regressão para garantir a consistência das mensagens de erro.

Link da publicação OpenCode: <https://opncd.ai/share/WRRypDlp>

Print consumo de token:

Revisão 1

Contexto

+

Sessão

Refatoração de branch excessivo em Python

Provedor

OpenAI

Limite de Contexto

400.000

Uso

10%

Tokens de Saída

165

Tokens de Cache (leitura/escrita)

41.472 / 0

Mensagens do Assistente

18

Sessão Criada

28 de jun. de 2026, 13:51

Mensagens

20

Modelo

GPT-5.4 mini

Total de Tokens

41.986

Tokens de Entrada

267

Tokens de Raciocínio

82

Mensagens de Usuário

2

Custo Total

US\$ 0,00

Última Atividade

28 de jun. de 2026, 13:54

Detalhamento do Contexto

●

Usuário 0,4%

●

Assistente 11,6%

●

Chamadas de Ferramentas 87,6%

●

Outros 0,4%

Análise da refatoração: A refatoração valeu a pena porque a função original concentrava 16 caminhos de decisão ao gerenciar de forma síncrona múltiplas lógicas complexas de layout, como striping de zebras, linhas normais de dados, agrupamentos, summary rows e footnotes. A separação do código em helpers privados especializados tornou o módulo de renderização HTML significativamente mais legível, limpo e fácil de manter. O uso da LLM foi vantajoso por identificar precisamente os limites de isolamento das linhas de dados em relação à estrutura de grupos e por garantir que a ordem de montagem não sofresse alterações que quebrassem os snapshots.

3.3.17 Ocorrência Número 17

Tabela de identificação

module	obj	message
great_tables._utils_render_html	_create_row_component_h	Too many branches (33/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint na função privada `_create_row_component_h`, localizada em `great_tables/_utils_render_html.py`. A LLM extraiu de forma cirúrgica os fluxos de decisão concorrentes para métodos auxiliares privados específicos por responsabilidade: `_create_summary_stub_cells_h` (para tratar stubs de linhas de resumo), `_get_row_cell_content_h` (para resolver o conteúdo bruto da célula) e `_get_row_cell_footnotes_h` (para isolar a lógica de busca e vinculação de notas de rodapé). Essa estratégia pulverizou os 33 caminhos de decisão originais, transformando a função principal em uma orquestradora linear e limpa, e preservou integralmente o contrato interno e o comportamento esperado da renderização do HTML.

Link da publicação OpenCode: <https://opncd.ai/share/IIArAFx9>

Print consumo de token:

Revisão 1	Contexto ×	+
Sessão	Mensagens	
Pylint too-many-branches em _create_row_component_h	26	
Provedor	Modelo	
OpenAI	GPT-5.4 mini	
Límite de Contexto	Total de Tokens	
400.000	41.363	
Uso	Tokens de Entrada	
10%	667	
Tokens de Saída	Tokens de Raciocínio	
199	49	
Tokens de Cache (leitura/escrita)	Mensagens de Usuário	
40.448 / 0	2	
Mensagens do Assistente	Custo Total	
24	US\$ 0,00	
Sessão Criada	Última Atividade	
28 de jun. de 2026, 14:00	28 de jun. de 2026, 14:05	
Detalhamento do Contexto		
		
● Usuário 0,6% ● Assistente 17,4% ● Chamadas de Ferramentas 81,7% ● Outros 0,3%		

Análise da refatoração: A refatoração valeu muito a pena. O método original apresentava uma complexidade ciclomática extremamente alta, pois centralizava simultaneamente regras de negócios para linhas normais de dados, linhas de resumo de grupo e grand summary, além de lógicas de estilização CSS e de filtragem de footnotes específicas para cada tipo de localização (loc). A separação do código em helpers menores tornou o módulo de renderização consideravelmente mais inteligível,

elegante e seguro contra regressões. O uso da LLM foi vantajoso por identificar as divisões exatas de escopo e por insistir na aplicação por blocos após a falha inicial de contexto do patch. Diferente das interações passadas, a LLM demonstrou excelente proatividade ao emular com sucesso um smoke test real de renderização (`GT(exibble).as_raw_html()`) logo após a verificação do `py_compile`, comprovando a integridade funcional da entrega.

3.3.18 Ocorrência Número 18

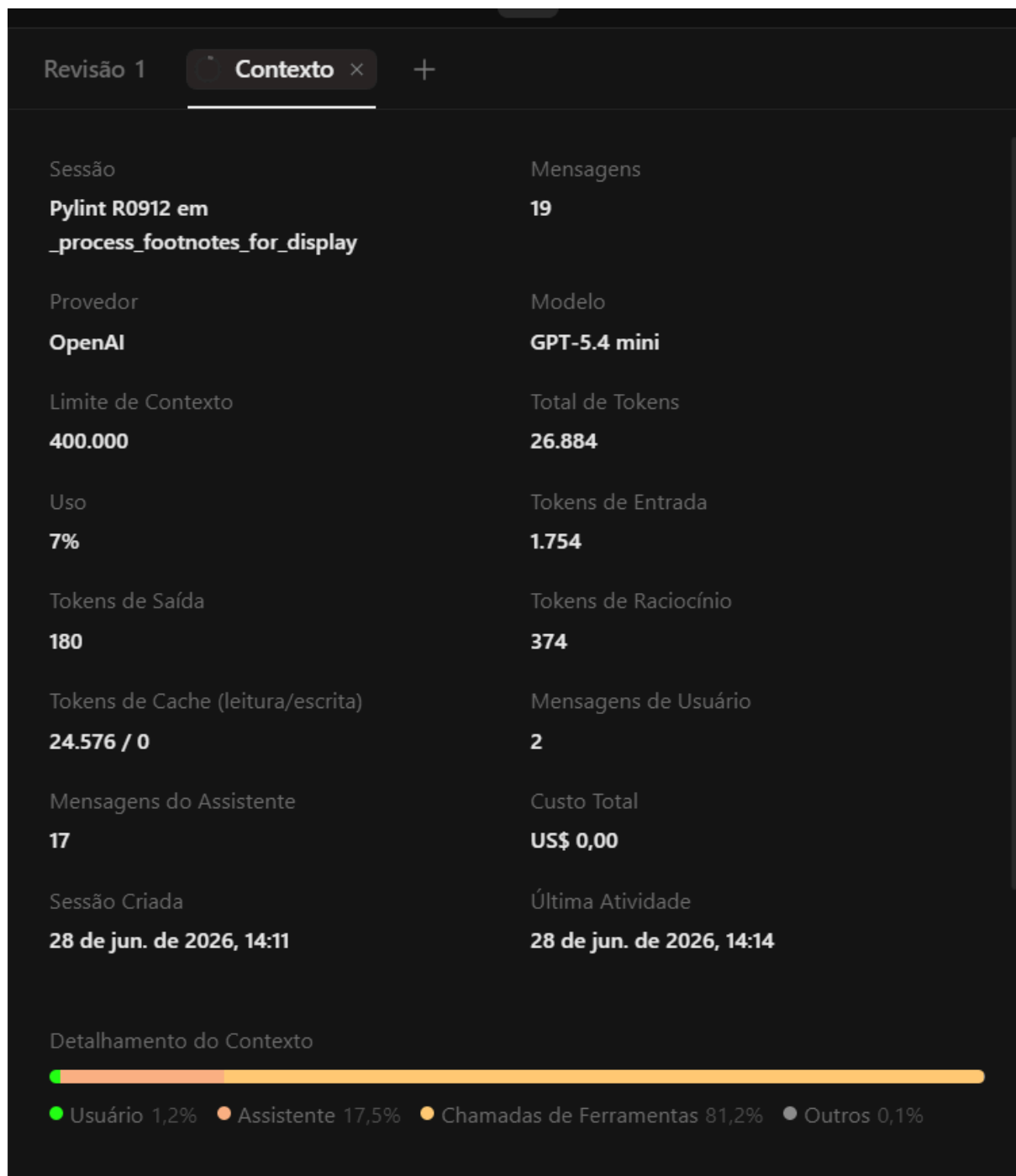
Tabela de identificação

module	obj	message
great_tables._utils_render_html	_process_footnotes_for_display	Too many branches (14/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint no método privado `_process_footnotes_for_display`, localizado em `great_tables/_utils_render_html.py`. A LLM extraiu de forma cirúrgica os fluxos de decisão paralelos para métodos auxiliares privados focados em responsabilidades únicas: `_get_footnote_display_sort_key` (para isolar os múltiplos ramos da lógica de ordenação e ordenação de chaves) e `_build_footnotes_for_display` (para encapsular a deduplicação e a montagem do retorno final). Essa estratégia pulverizou os 14 caminhos de decisão originais da função principal, transformando-a em uma orquestradora limpa responsável apenas pela filtragem preliminar, e preservou integralmente o contrato e o comportamento do sistema.

Link da publicação OpenCode: <https://opnecd.ai/share/GsukJcLC>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função original concentrava regras de naturezas muito distintas em uma única sequência lógica, misturando filtragem de visibilidade, ordenação complexa por chaves condicionais e deduplicação estrutural de notas de rodapé marcadas. A separação do código em helpers privados especializados tornou o módulo de renderização HTML muito mais inteligível e menos propenso a falhas durante futuras manutenções

na estrutura de rodapés da tabela. O uso da LLM foi vantajoso por identificar precisamente as fatias de código que inflavam a complexidade ciclomática sem causar impactos colaterais.

3.3.19 Ocorrência Número 19

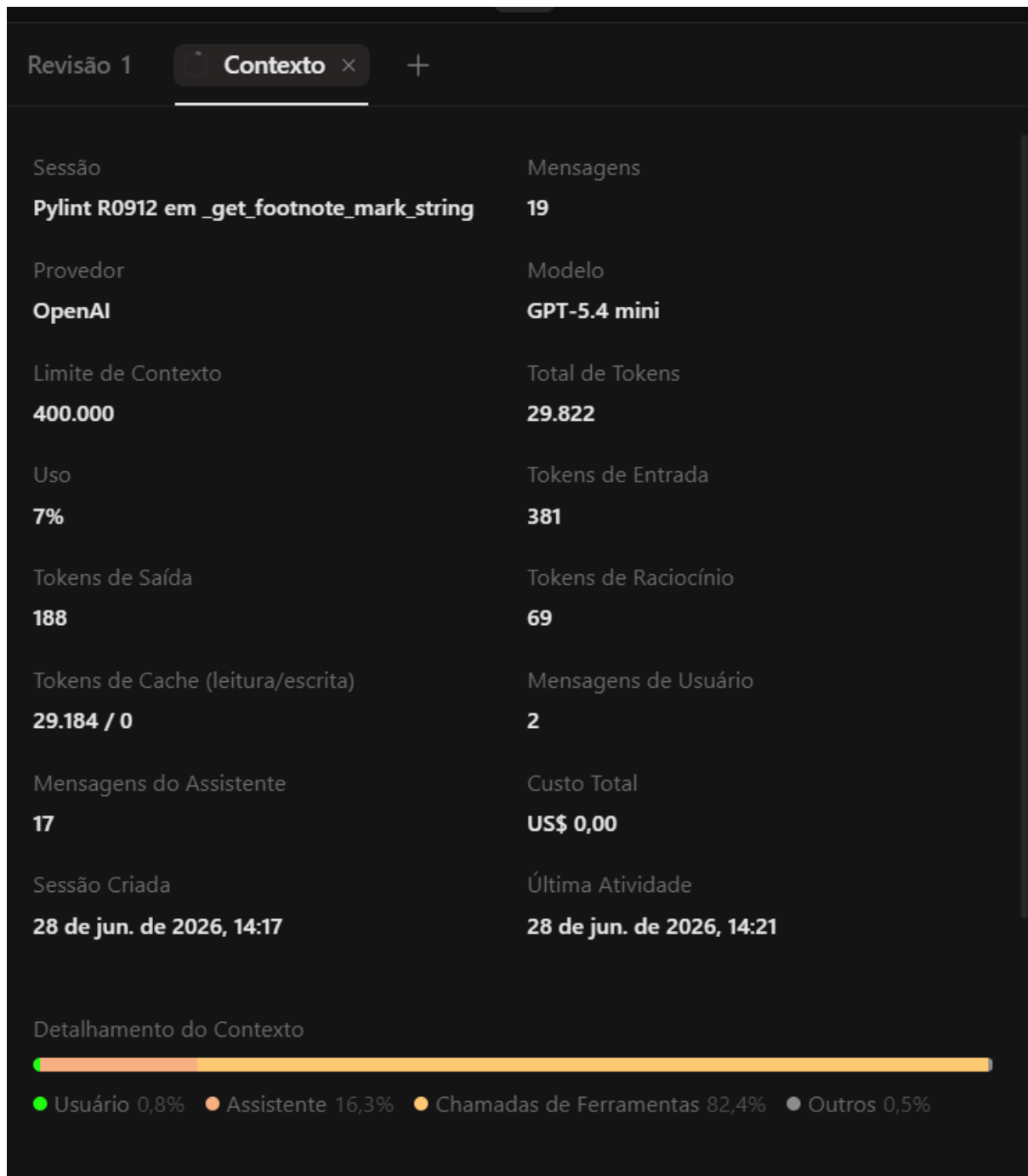
Tabela de identificação

module	obj	message
great_tables._utils_render_html	_get_footnote_mark_string	Too many branches (14/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint no método privado `_get_footnote_mark_string`, localizado em `great_tables/_utils_render_html.py`. A LLM extraiu de forma cirúrgica os fluxos de decisão paralelos para métodos auxiliares privados focados em responsabilidades únicas: `_get_footnote_position_key` (para isolar a montagem da chave de ordenação), `_get_footnote_colnum` (para encapsular a definição de `colnum` com base nas variantes de `locname`) e `_get_footnote_rownum` (para isolar a definição de `rownum`). Essa estratégia removeu a densa árvore de decisões e o bloco `try/except` da função principal, transformando-a em uma orquestradora linear e limpa, e preservou integralmente o contrato interno e o comportamento de resolução de marcadores de nota de rodapé.

Link da publicação OpenCode: <https://opnecd.ai/share/dFx6e7y2>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função original misturava regras de naturezas muito distintas em um único escopo de execução, acumulando filtros condicionais por tipo de localização, cálculo manual de índices de matriz e capturas de exceção para controle de fallback. A separação do código em helpers privados especializados tornou o módulo de renderização HTML significativamente mais legível, elegante e menos suscetível a erros de regressão na ordenação dos marcadores. O uso da LLM foi vantajoso por identificar precisamente as fatias de código responsáveis

por inflar a complexidade ciclomática e por aplicar a substituição do try/except por um fallback explícito mais idiomático.

3.3.20 Ocorrência Número 20

Tabela de identificação

module	obj	message
great_tables._utils_render_latex	create_body_component_l	Too many branches (21/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint no método `create_body_component_l`, localizado em `great_tables/_utils_render_latex.py`. A LLM extrai com sucesso a lógica de montagem e processamento concorrente da tabela LaTeX para métodos auxiliares privados focados em responsabilidades únicas: `_create_body_row_cells_l` (para gerar as células da linha), `_append_summary_rows_l` (para isolar a lógica de emissão de summaries) e `_append_group_header_row_l` (para gerenciar cabeçalhos de grupos). Essa estratégia reduziu as 21 ramificações originais da função principal, transformando-a em uma orquestradora linear e limpa, e preservou integralmente o contrato e o comportamento esperado da renderização de corpos de tabela em LaTeX.

Link da publicação OpenCode: <https://opnecd.ai/share/OT67IHC1>

Print consumo de token:

Revisão 1

Contexto

Sessão

Refatorar branch excessiva em _utils_render_latex

Provedor

OpenAI

Limite de Contexto

400.000

Uso

9%

Tokens de Saída

170

Tokens de Cache (leitura/escrita)

36.352 / 0

Mensagens do Assistente

19

Sessão Criada

28 de jun. de 2026, 14:24

Mensagens

21

Modelo

GPT-5.4 mini

Total de Tokens

37.111

Tokens de Entrada

551

Tokens de Raciocínio

38

Mensagens de Usuário

2

Custo Total

US\$ 0,00

Última Atividade

28 de jun. de 2026, 14:27

Detalhamento do Contexto

●

Usuário 0,7%

●

Assistente 14,2%

●

Chamadas de Ferramentas 84,9%

●

Outros 0,2%

Análise da refatoração: A refatoração valeu a pena porque a função original concentrava regras de naturezas muito distintas em um único fluxo iterativo, gerenciando simultaneamente a estilização de stubs, detecção de quebras de grupo, inserção síncrona de sumários em múltiplos níveis e formatação de células de dados. A separação do código em helpers privados especializados tornou o módulo de renderização LaTeX significativamente mais legível, elegante e menos propenso a erros de regressão estrutural. O uso da LLM foi vantajoso por identificar precisamente as fatias de código

responsáveis por inflar a complexidade ciclomática e por insistir na aplicação incremental da mudança após a falha inicial de contexto do patch.

3.3.21 Ocorrência Número 21

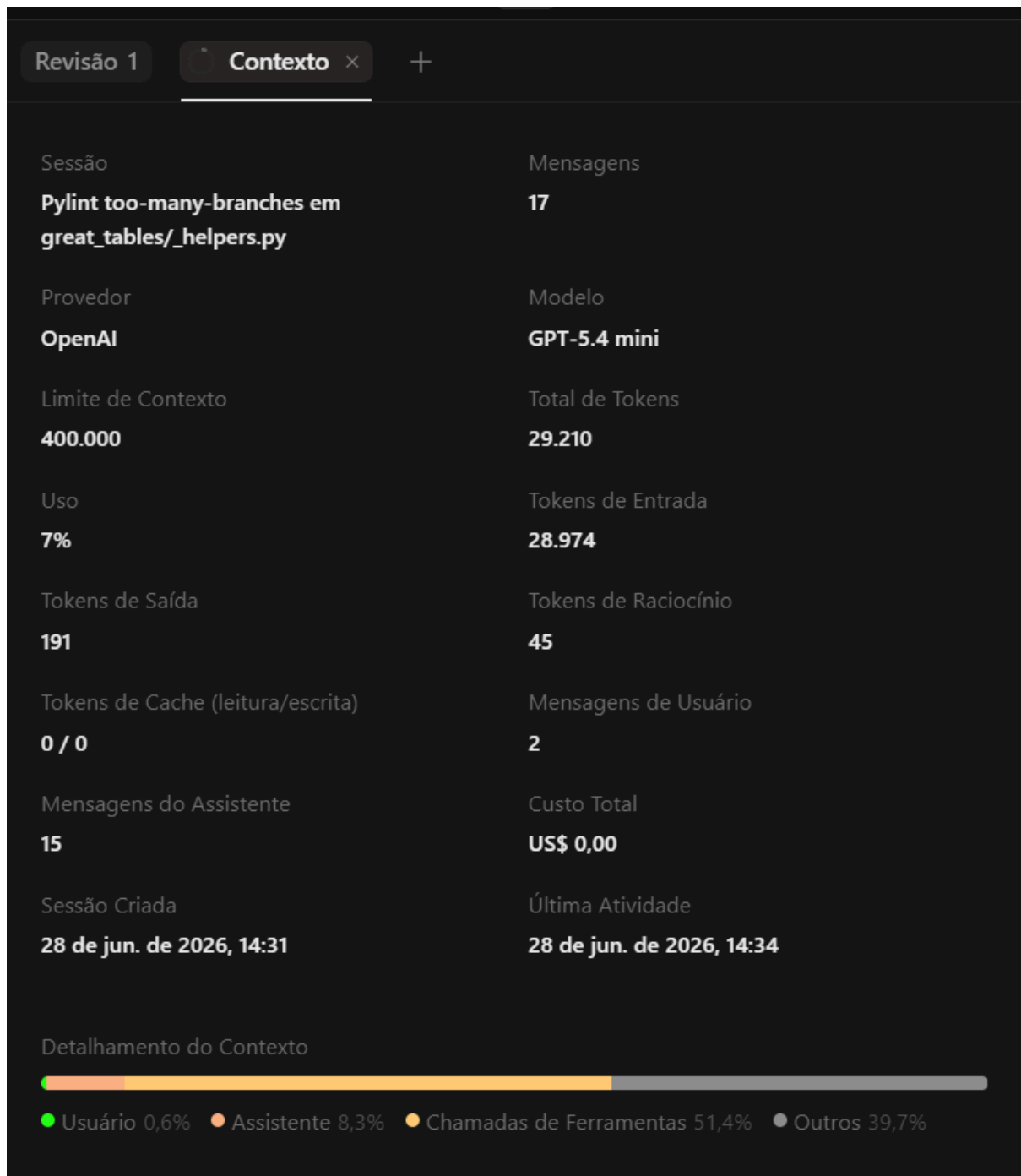
Tabela de identificação

module	obj	message
great_tables._helpers	UnitDefinition.to_html	Too many branches (14/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint no método `UnitDefinition.to_html`, localizado em `great_tables/_helpers.py`. A LLM extraiu com sucesso as responsabilidades de formatação de fragmentos de texto repetidos para os métodos auxiliares privados específicos: `_format_units_unit`, `_format_units_optional_superscript`, `_format_units_optional_subscript` e `_format_units_symbol_content`. Essa estratégia pulverizou os 14 caminhos de decisão originais, transformando o método principal em uma orquestração limpa e focada apenas nos casos especiais (fórmulas químicas e sobreposições), mantendo o contrato público e o comportamento do sistema intactos.

Link da publicação OpenCode: <https://opncd.ai/share/Qe8uhqHU>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque o método original acumulava múltiplas decisões de formatação textual concomitantes, misturando a substituição de símbolos, tratamento de sinais de subtração, marcações HTML e lógicas específicas para exponenciação e subscritos. A separação do código em helpers privados especializados eliminou a alta complexidade ciclômática e tornou a classe UnitDefinition muito mais modular, legível e protegida contra regressões. O uso da

LLM foi vantajoso por propor fatias cirúrgicas de corte de lógica e por garantir a consistência das saídas concatenadas.

3.3.22 Ocorrência Número 22

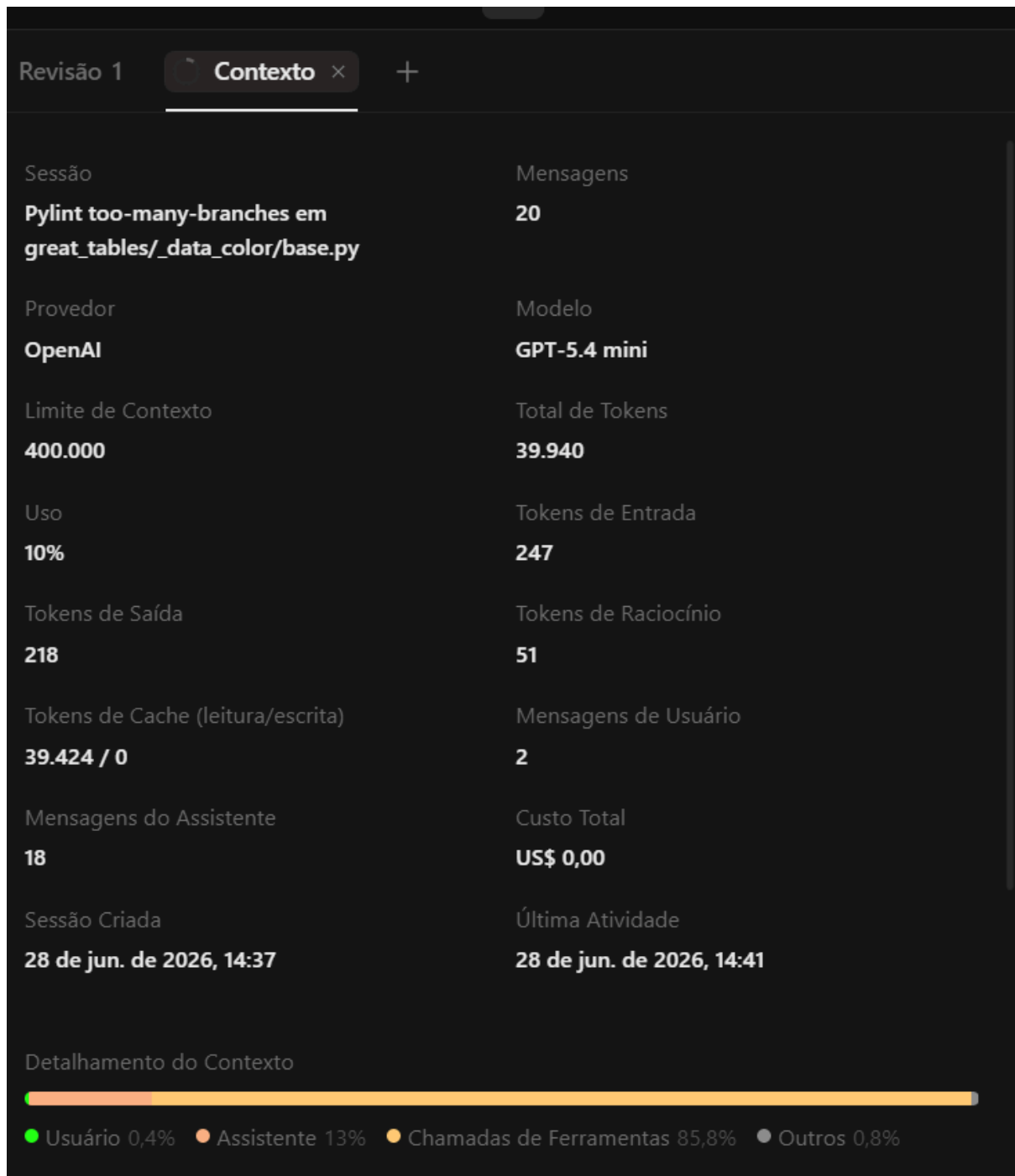
Tabela de identificação

module	obj	message
great_tables._data_color.base	data_color	Too many branches (16/12)

Seguiu o plano de refatoração: Sim. O plano de refatoração sugerido resolvia corretamente a ocorrência do code smell R0912 (too-many-branches) apontado pelo Pylint no método público `data_color()`, localizado em `great_tables/_data_color/base.py`. A LLM extraiu com sucesso a densa árvore de decisões em métodos auxiliares privados específicos por responsabilidade: `_resolve_na_color`, `_resolve_palette`, `_resolve_columns`, `_scale_column_values` e `_apply_column_styles`. Essa estratégia de pulverização isolou as 16 ramificações concorrentes da função principal, transformando-a em uma orquestradora linear e limpa, e preservou integralmente o contrato público e o comportamento esperado do sistema (incluindo as regras de recálculo de domínio por coluna).

Link da publicação OpenCode: <https://opncd.ai/share/CsNZOomd>

Print consumo de token:



Análise da refatoração: A refatoração valeu muito a pena. O método original apresentava uma alta complexidade ao misturar regras de normalização de parâmetros de paleta, filtragem de colunas e cálculos matemáticos iterativos de escala de cor por célula. A separação do código em helpers privados especializados tornou o módulo de colorização consideravelmente mais modular, legível e protegido contra regressões. O uso da LLM foi vantajoso por propor fatias exatas de isolamento de

lógica e por agir de forma corretiva durante a implementação, ajustando o escopo de importação de GradientPalette após detectar uma quebra de runtime no smoke test.

4. Métricas Após a Refatoração

4.1 Radon Após a Refatoração

4.1.1 Complexidade Ciclomática por Arquivo Após a Refatoração

Tabela de piores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
great_tables/_formats.py	145	4,24	23	615	D	_format_duration_colon_sep
great_tables/_utils_render_html.py	40	9,95	34	398	D	_create_columns_component_h_with h_spanners
great_tables/_utils_nanoplots.py	53	5,81	30	308	D	_prepare_nanoplot_state
great_tables/_gt_data.py	122	2,33	14	284	C	_collect_body_component_attrs
great_tables/_helpers.py	44	4,84	27	213	D	nanoplot_options

Tabela de melhores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
great_tables/_pipe.py	1	1,00	1	1	A	pipe
great_tables/_stub.py	1	1,00	1	1	A	reorder_stub_df
great_tables/_stubhead.py	1	1,00	1	1	A	tab_stubhead
great_tables/quarto.py	1	1,00	1	1	A	is_quarto_render
great_tables/_source_notes.py	1	1,00	1	1	A	tab_source_note

4.1.2 Complexidade Ciclomática por Função Após a Refatoração

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
great_tables/_utils_render_html.py	function	_create_columns_component_h_with _spanners	E	34
great_tables/_utils_render_html.py	function	_create_group_body_rows_h	E	31

great_tables/_utils_nanoplots.py	function	_prepare_nanoplot_state	D	30
great_tables/_utils_render_html.py	function	_get_row_cell_footnotes_h	D	30
great_tables/_helpers.py	function	nanoplot_options	D	27

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
great_tables/_body.py	function	body_reassemble	A	1
great_tables/_cols_merge.py	function	merge_pattern	A	1
great_tables/_data_color/base.py	function	_color_name_list	A	1
great_tables/_data_color/base.py	function	_get_wcag_contrast_ratio	A	1
great_tables/_data_color/base.py	function	_is_short_hex	A	1

4.1.3 Índice de Manutenibilidade Após a Refatoração

Tabela de piores métricas

arquivo	mi	rank_mi
great_tables/_gt_data.py	0,00	C
great_tables/_utils_nanoplots.py	0,00	C
great_tables/_formats.py	0,00	C
great_tables/_utils_render_html.py	0,00	C
great_tables/_spanners.py	6,64	C

Tabela de melhores métricas

arquivo	mi	rank_mi
great_tables/_row_groups.py	100,00	A
great_tables/vals.py	100,00	A
great_tables/_body.py	100,00	A
great_tables/_data_color/constants.py	100,00	A
great_tables/_data_color/init.py	100,00	A

4.1.4 Métricas Brutas Após a Refatoração

loc total	lloc total	sloc total	comments total	multi
32306	9303	15777	1652	9149

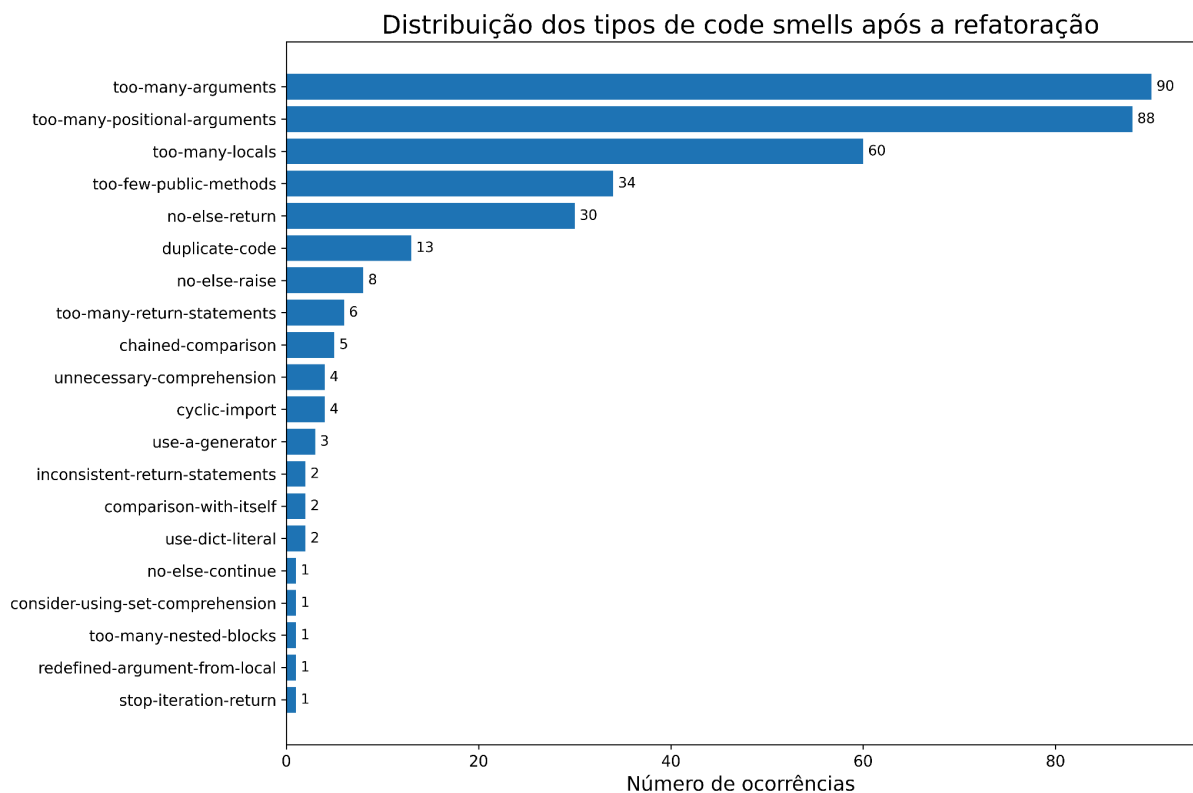
4.2 CodeCarbon Após a Refatoração

Preencha a tabela abaixo com as métricas ambientais depois da refatoração.

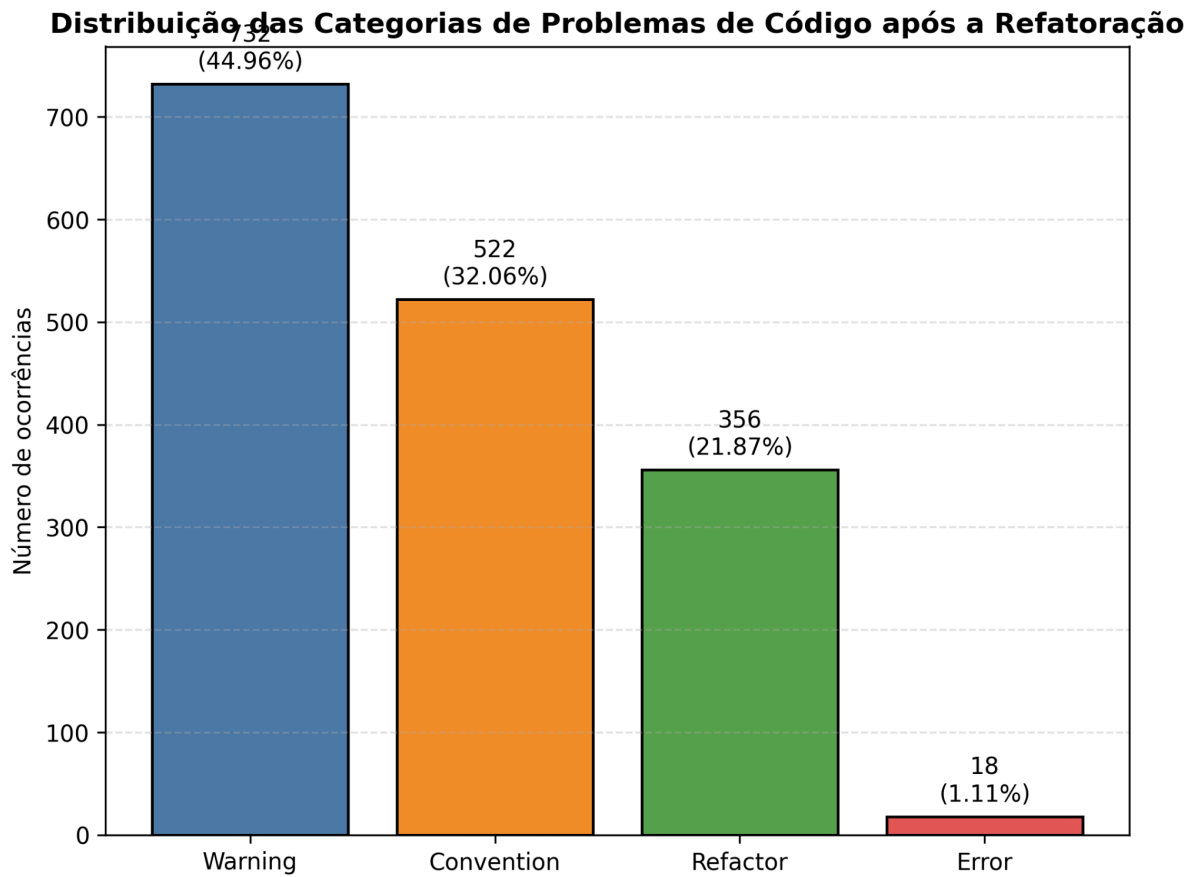
duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
1.57707 2700019 9258	6.773476013239639e-07	4.2949675136435106e-07	1.4364559082209382e-06	2.979962222161703e-06	6.887253440069589e-06

4.3 Pylint Após a Refatoração

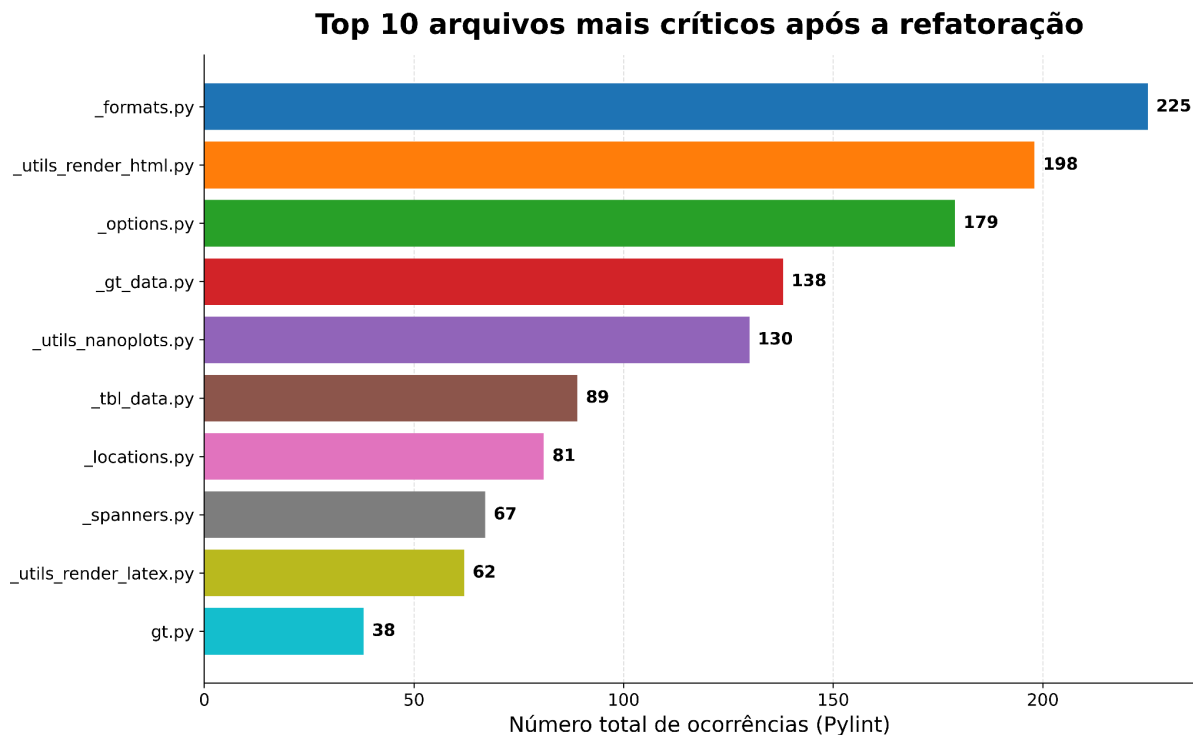
4.3.1 Distribuição do Smells Após a Refatoração



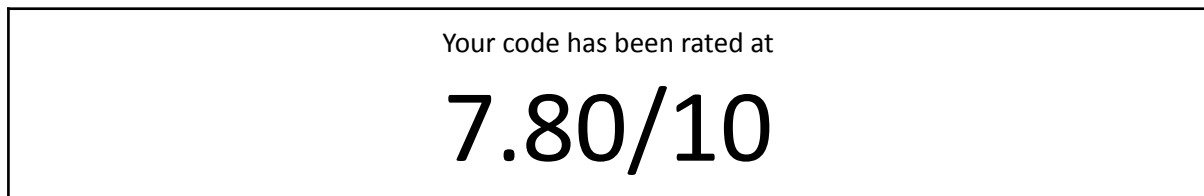
4.3.2 Distribuição dos Problemas de Código Após a Refatoração



4.3.3 Arquivos mais Críticos Após a Refatoração



4.3.4 Score Projeto Após a Refatoração



4.4 Pytest Após a Refatoração

4.4.1 Cobertura de testes Após a Refatoração

<i>File</i>	<i>Statements</i>	<i>Missing</i>	<i>Excluded</i>	<i>Coverage</i>
<i>great_tables/_init_.py</i>	16	2	0	87.50%
<i>great_tables/_body.py</i>	5	1	0	80.00%
<i>great_tables/_boxhead.py</i>	63	2	0	96.83%
<i>great_tables/_cols_merge.py</i>	136	71	0	47.80%

<i>File</i>	<i>Statements</i>	<i>Missing</i>	<i>Excluded</i>	<i>Coverage</i>
<i>great_tables/_databackend.py</i>	34	4	0	88.24%
<i>great_tables/_export.py</i>	140	98	0	30.00%
<i>great_tables/_footnotes.py</i>	24	5	0	79.17%
<i>great_tables/_formats.py</i>	98	54	0	44.90%
<i>great_tables/_formats_vals.py</i>	258	32	0	87.60%
<i>great_tables/_gt_data.py</i>	196	12	0	93.88%
<i>great_tables/_heading.py</i>	41	3	0	92.68%
<i>great_tables/_helpers.py</i>	323	204	0	36.84%
<i>great_tables/_locale.py</i>	321	82	0	74.45%
<i>great_tables/_locations.py</i>	179	34	0	81.01%
<i>great_tables/_modify_rows.py</i>	115	87	0	24.35%
<i>great_tables/_options.py</i>	442	41	0	90.72%
<i>great_tables/_pipe.py</i>	54	6	0	88.89%
<i>great_tables/_render.py</i>	124	108	0	12.90%
<i>great_tables/_render_checks.py</i>	52	8	0	84.62%
<i>great_tables/_row_groups.py</i>	83	24	0	71.08%
<i>great_tables/_scss.py</i>	248	157	0	36.69%
<i>great_tables/_source_notes.py</i>	24	4	0	83.33%
<i>great_tables/_spanners.py</i>	267	108	0	59.55%
<i>great_tables/_stub.py</i>	153	31	0	79.74%
<i>great_tables/_stubhead.py</i>	26	3	0	88.46%
<i>great_tables/_styles.py</i>	158	43	0	72.78%
<i>great_tables/_substitution.py</i>	185	134	0	27.57%
<i>great_tables/_tab_create_modify.py</i>	197	122	0	38.07%

<i>File</i>	<i>Statements</i>	<i>Missing</i>	<i>Excluded</i>	<i>Coverage</i>
<i>great_tables/_tbl_data.py</i>	358	162	0	54.75%
<i>great_tables/_tbl_data_align.py</i>	76	61	0	19.74%
<i>great_tables/_text.py</i>	33	4	0	87.88%
<i>great_tables/_types.py</i>	24	0	0	100.00%
<i>great_tables/_utils.py</i>	239	98	0	58.99%
<i>great_tables/_utils_nanoplots.py</i>	208	17	0	91.83%
<i>great_tables/_utils_render_html.py</i>	413	114	0	72.39%
<i>great_tables/_utils_render_latex.py</i>	115	88	0	23.48%
<i>great_tables/_utils_selenium.py</i>	45	32	0	28.89%
<i>great_tables/gt.py</i>	684	148	0	78.36%
<i>great_tables/loc.py</i>	35	0	0	100.00%
<i>great_tables/quarto.py</i>	56	31	0	44.64%
<i>great_tables/shiny.py</i>	32	19	0	40.63%
<i>great_tables/style.py</i>	25	0	0	100.00%
<i>great_tables/utils_render_common.py</i>	102	34	0	66.67%
<i>great_tables/vals.py</i>	20	0	0	100.00%
<i>great_tables/_data_color/__init__.py</i>	4	0	0	100.00%
<i>great_tables/_data_color/base.py</i>	296	185	0	37.50%
<i>great_tables/_data_color/constants.py</i>	12	0	0	100.00%
<i>great_tables/_data_color/palettes.py</i>	18	0	0	100.00%
<i>great_tables/data/__init__.py</i>	21	0	0	100.00%
<i>tests/__init__.py</i>	0	0	0	100.00%
<i>tests/conftest.py</i>	48	2	0	95.83%
<i>tests/test__boxhead.py</i>	25	0	0	100.00%

<i>File</i>	<i>Statements</i>	<i>Missing</i>	<i>Excluded</i>	<i>Coverage</i>
<i>tests/test__formats_vals.py</i>	54	0	0	100.00%
<i>tests/test__render_checks.py</i>	18	0	0	100.00%
<i>tests/test__stubhead.py</i>	14	0	0	100.00%
<i>tests/test__utils_nanoplots.py</i>	43	0	0	100.00%
<i>tests/test__utils_selenium.py</i>	14	4	0	71.43%
<i>tests/test_body.py</i>	10	0	0	100.00%
<i>tests/test_boxhead.py</i>	36	0	0	100.00%
<i>tests/test_cols_merge.py</i>	35	0	0	100.00%
<i>tests/test_datasets.py</i>	11	0	0	100.00%
<i>tests/test_dependencies.py</i>	16	1	0	93.75%
<i>tests/test_export.py</i>	32	0	0	100.00%
<i>tests/test_fmt_duration.py</i>	31	0	0	100.00%
<i>tests/test_fmt_partsper.py</i>	18	0	0	100.00%
<i>tests/test_footnotes.py</i>	53	0	0	100.00%
<i>tests/test_formats.py</i>	321	4	0	98.75%
<i>tests/test_formats_nanoplots.py</i>	38	0	0	100.00%
<i>tests/test_formats_vals.py</i>	168	1	0	99.40%
<i>tests/test_gt.py</i>	89	0	0	100.00%
<i>tests/test_gt_data.py</i>	31	0	0	100.00%
<i>tests/test_gtsave.py</i>	62	2	0	96.77%
<i>tests/test_gtsave_validation.py</i>	41	0	0	100.00%
<i>tests/test_heading.py</i>	29	0	0	100.00%
<i>tests/test_helpers.py</i>	86	0	0	100.00%
<i>tests/test_locations.py</i>	126	0	0	100.00%

<i>File</i>	<i>Statements</i>	<i>Missing</i>	<i>Excluded</i>	<i>Coverage</i>
<i>tests/test_modify_rows.py</i>	34	0	0	100.00%
<i>tests/test_nanoplots_snap.py</i>	18	0	0	100.00%
<i>tests/test_options.py</i>	129	0	0	100.00%
<i>tests/test_pipe.py</i>	18	0	0	100.00%
<i>tests/test_repr.py</i>	31	0	0	100.00%
<i>tests/test_row_groups.py</i>	36	0	0	100.00%
<i>tests/test_scss.py</i>	42	0	0	100.00%
<i>tests/test_shiny.py</i>	18	10	0	44.44%
<i>tests/test_spanners.py</i>	82	0	0	100.00%
<i>tests/test_styles.py</i>	112	0	0	100.00%
<i>tests/test_substitutions.py</i>	71	0	0	100.00%
<i>tests/test_summary_rows_snap.py</i>	22	0	0	100.00%
<i>tests/test_tab_create_modify.py</i>	34	0	0	100.00%
<i>tests/test_tbl_data.py</i>	124	0	0	100.00%
<i>tests/test_tbl_data_align.py</i>	19	0	0	100.00%
<i>tests/test_text.py</i>	15	0	0	100.00%
<i>tests/test_utils.py</i>	148	0	0	100.00%
<i>tests/test_utils_render_common.py</i>	23	0	0	100.00%
<i>tests/test_utils_render_html.py</i>	134	0	0	100.00%
<i>tests/test_utils_render_latex.py</i>	34	0	0	100.00%
<i>tests/utlis.py</i>	31	11	0	64.52%
<i>tests/data_color/test_data_color.py</i>	172	0	0	100.00%
<i>tests/data_color/test_data_color_utils.py</i>	68	0	0	100.00%

4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração

Quantidade de testes que passaram: 997

Quantidade de testes que falharam: 1068

5. Comparação dos Resultados

Ao compararmos o projeto antes e após a refatoração dos smells, observamos uma evolução mista, porém positiva na manutenibilidade do projeto Great Tables. Pelo lado positivo, as ações de refatoração focadas nos três code smells alvo (too-many-statements, too-many-instance-attributes e too-many-branches) tiveram sucesso, visto que esses smells sumiram completamente da análise do Pylint. O Pylint, que inicialmente havia atribuído uma nota de 7.76/10 registrou uma melhora perceptível em sua pontuação (7.80/10). Funções antes consideradas críticas devido aos seus altos índices de complexidade ciclomática via Radon (como `_generate_nanoplot` com CC de 148 e `create_columns_component_h` com CC de 78) tiveram seus fluxos pulverizados e simplificados radicalmente por meio da extração de helpers especializados, com a ajuda da LLM do Opencode. Esse isolamento de escopo aumentou significativamente a modularidade e o índice de manutenibilidade (Maintainability Index) nas regiões modificadas. Entretanto, observou-se no que se refere aos testes automatizados via Pytest que houve uma severa mudança em termos de estabilidade. Enquanto o cenário inicial contava com 2.086 testes bem-sucedidos e apenas 3 falhas, a validação pós-refatoração indicou que apenas 997 testes passaram, acumulando 1.068 falhas. Esse expressivo volume decorre do fato de que, embora as funções tenham sido isoladas em termos estáticos e tenham compilado com sucesso, as profundas alterações estruturais quebraram o comportamento esperado ou invalidaram os acoplamentos internos existentes nos testes do projeto original. Em relação às emissões de carbono monitoradas pelo CodeCarbon, o tempo de processamento e a taxa de consumo de recursos mantiveram-se estáveis, sem causar impactos ecológicos negativos adicionais expressivos.

6. Percepções sobre a Prática

O desenvolvimento desta atividade prática de manutenção de software proporcionou uma imersão valiosa nos desafios inerentes à evolução de sistemas modernos e projetos reais de código aberto (open-source). Atuar sobre uma biblioteca estruturada como o Great Tables permitiu compreender que a refatoração vai muito além de meras correções estéticas de sintaxe. A experiência exigiu uma análise minuciosa de acoplamento, o desenho de novas abstrações que não quebrassem as interfaces públicas e a constante preocupação com a retrocompatibilidade da API. Lidar com uma grande base de código revelou quão complexo pode ser o ecossistema de dependências de teste e como pequenas alterações em classes de dados estruturais geram impactos em cascata.

A refatoração dos três tipos de smells abordados (too-many-statements, too-many-instance-attributes e too-many-branches) mostrou-se crucial para a sustentabilidade de longo prazo em projetos colaborativos. Funções sobrecarregadas de comandos e ramificações condicionais aumentam a carga cognitiva de novos contribuidores, dificultam a escrita de testes

unitários localizados e elevam exponencialmente o risco de introdução de bugs a cada alteração. Da mesma forma, classes com excesso de atributos de instância operam como acumuladores de estado inchados, violando o princípio da responsabilidade única. Além destes, outros smells não tratados na atividade merecem igual prioridade no futuro, com destaque para *too-many-arguments* e *too-many-locals*, que encabeçam a lista de problemas do projeto original. O principal benefício observado com a refatoração foi o ganho imediato de legibilidade e a separação clara entre a coordenação do fluxo de negócios e as regras de renderização específicas. Por outro lado, o não-benefício manifestou-se na fragilidade temporária introduzida nos testes, gerando um esforço subsequente considerável para readequar as asserções e simulações (mocks) à nova arquitetura fragmentada do código.

A utilização de Large Language Models (LLMs) revelou-se um acelerador de produtividade formidável, embora demande uma supervisão técnica rigorosa e analítica por parte do desenvolvedor. A LLM demonstrou uma excelente capacidade de interpretar os alertas emitidos pelo Pylint e mapear com precisão os pontos de alta complexidade ciclomática estrutural. Suas sugestões iniciais para modularização e criação de helpers privados foram altamente aplicáveis e pouparam o esforço manual de recortar e organizar blocos repetitivos de código. Contudo, o processo tendeu a se complicar quando as mudanças exigiam uma visão holística do projeto: em alguns momentos, a IA tentou adotar soluções incompletas ou excessivamente simplistas (apenas movendo blocos de lugar sem de fato reduzir as ramificações), necessitando de intervenções e correções de percurso disparadas pelo usuário. No saldo geral, os projetos open-source podem se beneficiar largamente do uso de LLMs para eliminação de code smells, uma vez que a IA não apenas acelera a escrita dos novos métodos auxiliares, mas também ajuda na documentação automática, sugerindo nomes claros para os parâmetros e auxiliando na geração inicial de novas estruturas de dados adjacentes necessárias para encapsular o estado das classes refatoradas.

O processo de contribuição evidenciou que a inserção de novos desenvolvedores em repositórios públicos de grande porte é marcada por uma curva de aprendizado acentuada e barreiras técnicas significativas. As principais dificuldades enfrentadas estavam relacionadas à configuração inicial do ambiente, à resolução de conflitos de dependências estritas de empacotamento e à execução confiável de suites completas de testes locais. Compreender o fluxo de trabalho estabelecido pelos mantenedores e os critérios rigorosos de aceitação de Pull Requests (PRs) reforçou a importância de manter contratos e assinaturas públicas estritamente intocadas. Para facilitar as contribuições futuras em comunidades abertas, seria de extrema utilidade que os projetos adotassem ambientes de desenvolvimento totalmente containerizados (como Docker ou DevContainers configurados) e disponibilizassem uma documentação de arquitetura de alto nível detalhando o papel de cada módulo, diminuindo o tempo gasto na engenharia reversa e permitindo que os esforços se concentrassem diretamente na melhoria contínua da qualidade do código.

7. Links

- Link do fork: <https://github.com/dabliu89/great-tables>
- Link do projeto: <https://github.com/posit-dev/great-tables>
- Link do PR : <https://github.com/posit-dev/great-tables/pull/846>

